Delivery Dynamics

CONTENTS PAGE

01	main.py	3
02	models/full_model/rhs_aav.py	6
03	models/full_model/simulator.py	7
04	models/full_model/state_vector.py	9
05	models/pbpk/lymphatic_absorption.py	11
06	models/pbpk/organ_distribution.py	14
07	models/bbb/bbb_transport.py	17
08	models/intracellular/aav_intracellular.py	20
09	models/editing/apoe_multisite_editing.py	23
10	models/editing/competitive_editing.py	29
11	models/editing/module5.py	33
12	models/editing/offtarget_panel.py	35
13	models/editing/puf_apobec_mechanistic.py	38
14	models/editing/sequence_to_kinetics.py	43
15	models/calibration/__init__.py	46
16	models/calibration/parameter_fit.py	47
17	models/calibration/wetlab_bridge.py	50
18	models/__init__.py	54
19	utils/config_loader.py	55
20	utils/io.py	56
21	utils/plotting.py	57
22	config/base_config.yaml	65
23	config/parameter_provenance.yaml	68
24	optimization/iterated/__init__.py	73





25	optimization/iterated/uncertainty.py	74
26	optimization/metrics.py	76
27	optimization/min_effective_dose.py	77
28	optimization/report_module6.py	79
29	optimization/therapeutic_window.py	81
30	scripts/run_design_uncertainty.py	82
31	scripts/run_iterated_analysis.py	86
32	benchmarks/run_benchmarks.py	93
33	run_module6_scan.py	99
34	run_sensitivity_analysis.py	100
35	smoke_check.py	103
36	tests/test_administration_routes.py	104
37	tests/test_apoe_multisite_model.py	105
38	tests/test_brain_app.py	106
39	tests/test_offtarget_panel.py	110
40	tests/test_sequence_to_kinetics.py	111
41	tests/test_wetlab_bridge.py	112
42	requirements-core-tested.txt	113
43	requirements.txt	114
44	wetlab/templates/assay_observations_template.csv	115
45	wetlab/templates/offtarget_candidate_panel_template.csv	116
46	wetlab/templates/puf_design_candidates_template.csv	117





main.py

```python
from __future__ import annotations

import argparse
from pathlib import Path

from utils.config_loader import load_base_config
from utils.io import save_results, ensure_dir
from utils.plotting import plot_all
from models.full_model.simulator import run_simulation


def parse_args() -> argparse.Namespace:
    parser = argparse.ArgumentParser(
        description="AAV dynamics model: single simulation or module 6 dose optimization."
    )

    parser.add_argument(
        "--route",
        type=str,
        default=None,
        choices=["footpad", "iv", "im", "transfection"],
        help="Administration route.",
    )
    parser.add_argument(
        "--dose",
        type=float,
        default=1.0,
        help="Initial administered AAV amount for single simulation.",
    )
    parser.add_argument(
        "--t_end", type=float, default=None, help="Simulation end time in hours."
    )
    parser.add_argument(
        "--dt", type=float, default=None, help="Simulation output time step."
    )
    parser.add_argument(
        "--output_dir",
        type=str,
        default="outputs/run_full_model",
        help="Directory for single-simulation figures and saved results.",
    )

    parser.add_argument(
        "--optimize",
        action="store_true",
        help="Run module 6 dose optimization instead of a single simulation.",
    )
    parser.add_argument(
        "--scan_output_dir",
        type=str,
        default="outputs/run_module6",
        help="Directory for dose-scan CSV and optimization figures.",
    )

    return parser.parse_args()

```





```
57
58  def run_single_simulation(
59      project_root: Path,
60      config: dict,
61      dose: float,
62      t_end: float,
63      dt: float,
64      output_dir: str,
65  ) -> None:
66      results = run_simulation(
67          config=config,
68          dose=dose,
69          t_end=t_end,
70          dt=dt,
71      )
72
73      out_dir = ensure_dir(project_root / output_dir)
74      figure_paths = plot_all(results, out_dir)
75      save_results(results, out_dir)
76
77      print("Single simulation finished.")
78      print(f"Route: {config['route']}")
79      print(f"Dose: {dose}")
80      print(f"t_end: {t_end}")
81      print(f"dt: {dt}")
82      print(f"Output directory: {out_dir}")
83
84      if not results.get("success", True):
85          print(
86              f"WARNING: solver reported failure: {results.get('message', 'Unknown error')}"
87          )
88
89      print("\nGenerated figures:")
90      for name, path in figure_paths.items():
91          print(f"  {name}: {path}")
92
93
94  def run_optimization(
95      project_root: Path,
96      config: dict,
97      t_end: float,
98      dt: float,
99      scan_output_dir: str,
100 ) -> None:
101     from optimization.min_effective_dose import run_dose_scan, find_min_effective_dose
102     from optimization.report_module6 import save_dose_scan_table, plot_dose_response
103
104     scan_results = run_dose_scan(
105         base_config=config,
106         t_end=t_end,
107         dt=dt,
108     )
109     best = find_min_effective_dose(scan_results)
110
111     scan_out_dir = ensure_dir(project_root / scan_output_dir)
112     csv_path = save_dose_scan_table(scan_results, scan_out_dir)
113     fig_paths = plot_dose_response(scan_results, scan_out_dir)
114
115     print("Dose optimization finished.")
116     print(f"Route: {config['route']}")
117     print(f"t_end: {t_end}")
```





```
    print(f"dt: {dt}")
    print(f"Scan output directory: {scan_out_dir}")
    print(f"Dose scan saved to: {csv_path}")

    if best is None:
        print("\nNo feasible dose found under current thresholds.")
    else:
        print("\nMinimum feasible dose found:")
        for k, v in best.items():
            print(f"  {k}: {v}")

    print("\nGenerated optimization figures:")
    for name, path in fig_paths.items():
        print(f"  {name}: {path}")


def main() -> None:
    args = parse_args()
    project_root = Path(__file__).resolve().parent

    # Load base configuration
    config = load_base_config(project_root)

    # Override route if provided
    if args.route is not None:
        config["route"] = args.route

    # Resolve simulation settings EARLY so both branches can use them
    t_end = (
        args.t_end if args.t_end is not None else config["simulation"]["default_t_end"]
    )
    dt = args.dt if args.dt is not None else config["simulation"]["default_dt"]

    if args.optimize:
        run_optimization(
            project_root=project_root,
            config=config,
            t_end=t_end,
            dt=dt,
            scan_output_dir=args.scan_output_dir,
        )
    else:
        run_single_simulation(
            project_root=project_root,
            config=config,
            dose=args.dose,
            t_end=t_end,
            dt=dt,
            output_dir=args.output_dir,
        )


if __name__ == "__main__":
    main()
```





models/full_model/rhs_aav.py

```python
import numpy as np

from models.pbpk.lymphatic_absorption import lymphatic_absorption_rhs
from models.pbpk.organ_distribution import organ_distribution_rhs
from models.bbb.bbb_transport import bbb_transport_rhs
from models.intracellular.aav_intracellular import aav_intracellular_rhs
from models.editing.module5 import editing_rhs


def rhs_aav(t, y, config, idx):
    """
    Total RHS for current AAV model:
    Module 1: absorption
    Module 2: systemic distribution
    Module 3: BBB transport
    Module 4: intracellular uptake and expression
    Module 5: competitive editing
    """
    dydt = np.zeros_like(y)

    # Module 1 (skipped for in-vitro transfection mode: no absorption depot)
    absorption_cfg = config.get("absorption", {})
    if config["route"] in absorption_cfg:
        dydt += lymphatic_absorption_rhs(t, y, absorption_cfg[config["route"]], idx)

    # Module 2
    distribution_params = config["distribution"]
    dydt += organ_distribution_rhs(t, y, distribution_params, idx)

    # Module 3
    bbb_params = config["bbb"]
    dydt += bbb_transport_rhs(t, y, bbb_params, idx)

    # Module 4
    intracellular_params = config["intracellular"]
    dydt += aav_intracellular_rhs(t, y, intracellular_params, idx)

    # Module 5
    editing_params = config["editing"]
    dydt += editing_rhs(t, y, editing_params, idx)

    return dydt
```





models/full_model/simulator.py

```python
import numpy as np
from scipy.integrate import solve_ivp

from models.full_model.state_vector import STATE_ORDER, build_index_map
from models.full_model.rhs_aav import rhs_aav


def build_initial_state(dose: float, config) -> np.ndarray:
    """
    Build the global initial state vector.
    """
    idx = build_index_map(STATE_ORDER)
    y0 = np.zeros(len(STATE_ORDER), dtype=float)

    # Module 1 initial input
    route = config.get("route")
    if route == "iv":
        y0[idx["A_blood"]] = dose
    elif route == "transfection":
        # HEK293T transient-transfection mode: plasmid dose enters the nuclear
        # expression compartment directly, bypassing Modules 1–3 (delivery).
        y0[idx["A_brain_nuc"]] = dose
    else:
        y0[idx["A_dep"]] = dose

    # Module 5 initial substrate pools
    editing_model = config["editing"].get("model", "competitive")
    if editing_model == "apoe_multisite":
        editing = config["editing"]
        y0[idx["S_APOE4"]] = editing["S_APOE4_init"]
        y0[idx["S_puf_off"]] = editing["S_puf_off_init"]
        y0[idx["S_deaminase_bg"]] = editing["S_deaminase_bg_init"]
    else:
        y0[idx["S_on"]] = config["editing"]["S_on_init"]
        y0[idx["S_off"]] = config["editing"]["S_off_init"]

    return y0


def run_simulation(config, dose=1.0, t_end=72.0, dt=0.1):
    """
    Run the current AAV dynamics simulation.
    """
    idx = build_index_map(STATE_ORDER)
    y0 = build_initial_state(dose, config)

    t_eval = np.arange(0.0, t_end + dt, dt)

    sol = solve_ivp(
        fun=lambda t, y: rhs_aav(t, y, config, idx),
        t_span=(0.0, t_end),
        y0=y0,
        t_eval=t_eval,
        method="LSODA",
        rtol=1e-6,
        atol=1e-9,
```





```
    )

    return {
        "t": sol.t,
        "y": sol.y,
        "idx": idx,
        "state_order": STATE_ORDER,
        "dose": dose,
        "success": sol.success,
        "message": sol.message,
        "config": config,
    }
```





models/full_model/state_vector.py

```python
from __future__ import annotations

from typing import Dict, List, Tuple
import numpy as np

STATE_ORDER: List[str] = [
    "A_dep",
    "A_lymph",
    "A_blood",
    "A_liver",
    "A_peripheral",
    "A_brain_blood",
    "A_cleared",
    "A_brain_EC",
    "A_brain_endo",
    "A_brain_ISF",
    "A_brain_cell",
    "A_brain_nuc",
    "mRNA_brain",
    "P_brain",
    "S_on",
    "S_off",
    "ES_on",
    "ES_off",
    "E_on",
    "E_off",
    "S_APOE4",
    "S_APOE3_like",
    "S_APOE2_like",
    "S_APOE158_only",
    "C_APOE112",
    "C_APOE158",
    "C_APOE158_after112",
    "C_APOE112_after158",
    "B_local_bystander",
    "S_puf_off",
    "C_puf_off",
    "E_puf_off",
    "S_deaminase_bg",
    "E_deaminase_bg",
]


def build_index_map(state_order: List[str] | None = None) -> Dict[str, int]:
    """Build a name -> index mapping for the global state vector."""
    order = state_order or STATE_ORDER
    return {name: i for i, name in enumerate(order)}


IDX: Dict[str, int] = build_index_map()


def make_initial_state(dose: float) -> np.ndarray:
    """
    Create the default initial condition vector.

```





```
    Parameters
    ----------
    dose : float
        Initial administered AAV amount placed in the depot compartment.
    """
    y0 = np.zeros(len(STATE_ORDER), dtype=float)
    y0[IDX["A_dep"]] = dose
    return y0


def as_state_dict(y: np.ndarray, idx: Dict[str, int] | None = None) -> Dict[str, float]:
    """Convert a state vector into a readable dictionary."""
    index_map = idx or IDX
    return {name: float(y[i]) for name, i in index_map.items()}


__all__ = [
    "STATE_ORDER",
    "IDX",
    "build_index_map",
    "make_initial_state",
    "as_state_dict",
]
```





models/pbpk/lymphatic_absorption.py

```python
from __future__ import annotations

from typing import Dict
import numpy as np


REQUIRED_ABSORPTION_KEYS = (
    "k_lymph",
    "k_blood",
    "k_deg_loc",
    "k_lymph_to_blood",
)


def get_absorption_params(config: Dict, route: str) -> Dict[str, float]:
    """
    Extract route-specific absorption parameters from the global configuration.
    """
    if "absorption" not in config:
        raise KeyError("Missing 'absorption' section in config.")
    if route not in config["absorption"]:
        raise KeyError(f"Unsupported route '{route}' in absorption config.")
    params = config["absorption"][route]
    validate_absorption_params(params)
    return params


def validate_absorption_params(params: Dict[str, float]) -> None:
    """Validate that all absorption parameters exist and are non-negative."""
    for key in REQUIRED_ABSORPTION_KEYS:
        if key not in params:
            raise KeyError(f"Missing absorption parameter: {key}")
        if params[key] < 0:
            raise ValueError(f"Absorption parameter '{key}' must be non-negative.")


def compute_absorption_fluxes(
    A_dep: float,
    A_lymph: float,
    params: Dict[str, float],
) -> Dict[str, float]:
    """
    Compute the four physical fluxes in module 1.

    Returns
    -------
    dict
        Keys:
        - to_lymph
        - to_blood
        - local_loss
        - lymph_to_blood
    """
    validate_absorption_params(params)

    k_lymph = params["k_lymph"]
```





```
    k_blood = params["k_blood"]
    k_deg_loc = params["k_deg_loc"]
    k_lymph_to_blood = params["k_lymph_to_blood"]

    return {
        "to_lymph": k_lymph * A_dep,
        "to_blood": k_blood * A_dep,
        "local_loss": k_deg_loc * A_dep,
        "lymph_to_blood": k_lymph_to_blood * A_lymph,
    }


def lymphatic_absorption_rhs(
    t: float,
    y: np.ndarray,
    params: Dict[str, float],
    idx: Dict[str, int],
) -> np.ndarray:
    """
    RHS contribution of the absorption module.

    This module governs:
        depot -> lymph
        depot -> blood
        depot -> local loss
        lymph -> blood
    """
    _ = t  # reserved for future time-dependent extensions

    dydt = np.zeros_like(y)

    A_dep = y[idx["A_dep"]]
    A_lymph = y[idx["A_lymph"]]

    fluxes = compute_absorption_fluxes(A_dep=A_dep, A_lymph=A_lymph, params=params)

    to_lymph = fluxes["to_lymph"]
    to_blood = fluxes["to_blood"]
    local_loss = fluxes["local_loss"]
    lymph_to_blood = fluxes["lymph_to_blood"]

    dydt[idx["A_dep"]] += -(to_lymph + to_blood + local_loss)
    dydt[idx["A_lymph"]] += to_lymph - lymph_to_blood
    dydt[idx["A_blood"]] += to_blood + lymph_to_blood

    return dydt


def extract_absorption_flux_trajectories(
    x_array: np.ndarray,
    y_array: np.ndarray,
    params: Dict[str, float],
    idx: Dict[str, int],
) -> Dict[str, np.ndarray]:
    """
    Reconstruct module-1 fluxes from a simulated trajectory.

    Parameters
    ----------
    y_matrix : np.ndarray
        Shape (n_states, n_times).
```





```
    params : dict
        Absorption parameter dictionary.
    idx : dict
        State index map.
    """
    flux_history = {
        "to_lymph": [],
        "to_blood": [],
        "local_loss": [],
        "lymph_to_blood": [],
    }

    for k in range(y_array.shape[1]):
        A_dep = float(y_array[idx["A_dep"], k])
        A_lymph = float(y_array[idx["A_lymph"], k])
        fluxes = compute_absorption_fluxes(A_dep=A_dep, A_lymph=A_lymph, params=params)
        for name in flux_history:
            flux_history[name].append(fluxes[name])

    for name in flux_history:
        flux_history[name] = np.asarray(flux_history[name], dtype=float)

    return flux_history


__all__ = [
    "get_absorption_params",
    "validate_absorption_params",
    "compute_absorption_fluxes",
    "lymphatic_absorption_rhs",
    "extract_absorption_flux_trajectories",
]
```





models/pbpk/organ_distribution.py

```python
from __future__ import annotations

from typing import Dict
import numpy as np


def validate_distribution_params(params: Dict[str, float]) -> None:
    required = [
        "k_blood_to_liver",
        "k_blood_to_peripheral",
        "k_blood_to_brain",
        "k_liver_to_blood",
        "k_peripheral_to_blood",
        "k_brain_to_blood",
        "k_clear_blood",
        "k_clear_liver",
        "k_clear_peripheral",
        "k_clear_brainblood",
    ]
    for key in required:
        if key not in params:
            raise KeyError(f"Missing distribution parameter: {key}")
        if params[key] < 0:
            raise ValueError(f"Distribution parameter '{key}' must be non-negative.")


def compute_distribution_fluxes(
    A_blood: float,
    A_liver: float,
    A_peripheral: float,
    A_brain_blood: float,
    params: Dict[str, float],
) -> Dict[str, float]:
    """
    Compute all module-2 fluxes.
    """
    fluxes = {
        "blood_to_liver": params["k_blood_to_liver"] * A_blood,
        "blood_to_peripheral": params["k_blood_to_peripheral"] * A_blood,
        "blood_to_brain": params["k_blood_to_brain"] * A_blood,
        "liver_to_blood": params["k_liver_to_blood"] * A_liver,
        "peripheral_to_blood": params["k_peripheral_to_blood"] * A_peripheral,
        "brain_to_blood": params["k_brain_to_blood"] * A_brain_blood,
        "clear_blood": params["k_clear_blood"] * A_blood,
        "clear_liver": params["k_clear_liver"] * A_liver,
        "clear_peripheral": params["k_clear_peripheral"] * A_peripheral,
        "clear_brainblood": params["k_clear_brainblood"] * A_brain_blood,
    }
    return fluxes


def organ_distribution_rhs(
    t: float,
    y: np.ndarray,
    params: Dict[str, float],
    idx: Dict[str, int],
```





```
) -> np.ndarray:
    """
    Module 2: whole-body distribution
    blood <-> liver
    blood <-> peripheral
    blood <-> brain_blood
    + clearance
    """
    _ = t
    validate_distribution_params(params)

    dydt = np.zeros_like(y)

    A_blood = y[idx["A_blood"]]
    A_liver = y[idx["A_liver"]]
    A_peripheral = y[idx["A_peripheral"]]
    A_brain_blood = y[idx["A_brain_blood"]]

    fluxes = compute_distribution_fluxes(
        A_blood=A_blood,
        A_liver=A_liver,
        A_peripheral=A_peripheral,
        A_brain_blood=A_brain_blood,
        params=params,
    )

    # Blood
    dydt[idx["A_blood"]] += (
        -fluxes["blood_to_liver"]
        - fluxes["blood_to_peripheral"]
        - fluxes["blood_to_brain"]
        - fluxes["clear_blood"]
        + fluxes["liver_to_blood"]
        + fluxes["peripheral_to_blood"]
        + fluxes["brain_to_blood"]
    )

    # Liver
    dydt[idx["A_liver"]] += (
        fluxes["blood_to_liver"] - fluxes["liver_to_blood"] - fluxes["clear_liver"]
    )

    # Peripheral lumped compartment
    dydt[idx["A_peripheral"]] += (
        fluxes["blood_to_peripheral"]
        - fluxes["peripheral_to_blood"]
        - fluxes["clear_peripheral"]
    )

    # Brain vascular side
    dydt[idx["A_brain_blood"]] += (
        fluxes["blood_to_brain"] - fluxes["brain_to_blood"] - fluxes["clear_brainblood"]
    )

    # Cleared accumulator
    dydt[idx["A_cleared"]] += (
        fluxes["clear_blood"]
        + fluxes["clear_liver"]
        + fluxes["clear_peripheral"]
        + fluxes["clear_brainblood"]
    )
```





```

    return dydt


def extract_distribution_flux_trajectories(
    t_array: np.ndarray,
    y_array: np.ndarray,
    params: Dict[str, float],
    idx: Dict[str, int],
) -> Dict[str, np.ndarray]:
    """
    Reconstruct module-2 flux trajectories from simulation result.
    """
    flux_history = {
        "blood_to_liver": [],
        "blood_to_peripheral": [],
        "blood_to_brain": [],
        "liver_to_blood": [],
        "peripheral_to_blood": [],
        "brain_to_blood": [],
        "clear_blood": [],
        "clear_liver": [],
        "clear_peripheral": [],
        "clear_brainblood": [],
    }

    for k in range(y_array.shape[1]):
        A_blood = y_array[idx["A_blood"], k]
        A_liver = y_array[idx["A_liver"], k]
        A_peripheral = y_array[idx["A_peripheral"], k]
        A_brain_blood = y_array[idx["A_brain_blood"], k]

        fluxes = compute_distribution_fluxes(
            A_blood=A_blood,
            A_liver=A_liver,
            A_peripheral=A_peripheral,
            A_brain_blood=A_brain_blood,
            params=params,
        )

        for name in flux_history:
            flux_history[name].append(fluxes[name])

    for name in flux_history:
        flux_history[name] = np.asarray(flux_history[name], dtype=float)

    return flux_history
```





models/bbb/bbb_transport.py

```python
from __future__ import annotations

from typing import Dict
import numpy as np


def validate_bbb_params(params: Dict[str, float]) -> None:
    required = [
        "k_brainblood_to_EC",
        "k_EC_to_endo",
        "k_EC_to_brainblood",
        "k_endo_to_ISF",
        "k_endo_to_brainblood",
        "k_endo_loss",
        "k_clear_ISF",
    ]
    for key in required:
        if key not in params:
            raise KeyError(f"Missing BBB parameter: {key}")
        if params[key] < 0:
            raise ValueError(f"BBB parameter '{key}' must be non-negative.")


def compute_bbb_fluxes(
    A_brain_blood: float,
    A_brain_EC: float,
    A_brain_endo: float,
    A_brain_ISF: float,
    params: Dict[str, float],
) -> Dict[str, float]:
    """
    Compute all module-3 BBB transport fluxes.
    """
    fluxes = {
        "brainblood_to_EC": params["k_brainblood_to_EC"] * A_brain_blood,
        "EC_to_endo": params["k_EC_to_endo"] * A_brain_EC,
        "EC_to_brainblood": params["k_EC_to_brainblood"] * A_brain_EC,
        "endo_to_ISF": params["k_endo_to_ISF"] * A_brain_endo,
        "endo_to_brainblood": params["k_endo_to_brainblood"] * A_brain_endo,
        "endo_loss": params["k_endo_loss"] * A_brain_endo,
        "clear_ISF": params["k_clear_ISF"] * A_brain_ISF,
    }
    return fluxes


def bbb_transport_rhs(
    t: float,
    y: np.ndarray,
    params: Dict[str, float],
    idx: Dict[str, int],
) -> np.ndarray:
    """
    Module 3: BBB transport dynamics.

    States affected:
    - A_brain_blood
```





```
    - A_brain_EC
    - A_brain_endo
    - A_brain_ISF
    - A_cleared (for endo_loss + ISF clearance accumulation)
    """
    _ = t
    validate_bbb_params(params)

    dydt = np.zeros_like(y)

    A_brain_blood = y[idx["A_brain_blood"]]
    A_brain_EC = y[idx["A_brain_EC"]]
    A_brain_endo = y[idx["A_brain_endo"]]
    A_brain_ISF = y[idx["A_brain_ISF"]]

    fluxes = compute_bbb_fluxes(
        A_brain_blood=A_brain_blood,
        A_brain_EC=A_brain_EC,
        A_brain_endo=A_brain_endo,
        A_brain_ISF=A_brain_ISF,
        params=params,
    )

    # A_brain_blood:
    # lose to EC, gain from EC dissociation and endo recycling
    dydt[idx["A_brain_blood"]] += (
        -fluxes["brainblood_to_EC"]
        + fluxes["EC_to_brainblood"]
        + fluxes["endo_to_brainblood"]
    )

    # A_brain_EC:
    # gain from brain blood, lose to endo and back to brain blood
    dydt[idx["A_brain_EC"]] += (
        fluxes["brainblood_to_EC"] - fluxes["EC_to_endo"] - fluxes["EC_to_brainblood"]
    )

    # A_brain_endo:
    # gain from EC, lose to ISF, brainblood recycling, and degradation
    dydt[idx["A_brain_endo"]] += (
        fluxes["EC_to_endo"]
        - fluxes["endo_to_ISF"]
        - fluxes["endo_to_brainblood"]
        - fluxes["endo_loss"]
    )

    # A_brain_ISF:
    # gain from successful transcytosis, lose by local clearance
    # later module 4 will additionally remove from ISF via cell uptake
    dydt[idx["A_brain_ISF"]] += fluxes["endo_to_ISF"] - fluxes["clear_ISF"]

    # Cleared accumulator
    dydt[idx["A_cleared"]] += fluxes["endo_loss"] + fluxes["clear_ISF"]

    return dydt


def extract_bbb_flux_trajectories(
    t_array: np.ndarray,
    y_array: np.ndarray,
    params: Dict[str, float],
```





```
    idx: Dict[str, int],
) -> Dict[str, np.ndarray]:
    """
    Reconstruct module-3 BBB flux trajectories from simulation result.
    """
    flux_history = {
        "brainblood_to_EC": [],
        "EC_to_endo": [],
        "EC_to_brainblood": [],
        "endo_to_ISF": [],
        "endo_to_brainblood": [],
        "endo_loss": [],
        "clear_ISF": [],
    }

    for k in range(y_array.shape[1]):
        A_brain_blood = y_array[idx["A_brain_blood"], k]
        A_brain_EC = y_array[idx["A_brain_EC"], k]
        A_brain_endo = y_array[idx["A_brain_endo"], k]
        A_brain_ISF = y_array[idx["A_brain_ISF"], k]

        fluxes = compute_bbb_fluxes(
            A_brain_blood=A_brain_blood,
            A_brain_EC=A_brain_EC,
            A_brain_endo=A_brain_endo,
            A_brain_ISF=A_brain_ISF,
            params=params,
        )

        for name in flux_history:
            flux_history[name].append(fluxes[name])

    for name in flux_history:
        flux_history[name] = np.asarray(flux_history[name], dtype=float)

    return flux_history
```





models/intracellular/aav_intracellular.py

```python
from __future__ import annotations

from typing import Dict
import numpy as np


def validate_intracellular_params(params: Dict[str, float]) -> None:
    required = [
        "k_ISF_to_cell",
        "k_cell_to_nuc",
        "k_cell_loss",
        "k_deg_v",
        "k_tx",
        "k_deg_m",
        "k_tl",
        "k_deg_p",
    ]
    for key in required:
        if key not in params:
            raise KeyError(f"Missing intracellular parameter: {key}")
        if params[key] < 0:
            raise ValueError(f"Intracellular parameter '{key}' must be non-negative.")


def compute_intracellular_fluxes(
    A_brain_ISF: float,
    A_brain_cell: float,
    A_brain_nuc: float,
    mRNA_brain: float,
    P_brain: float,
    params: Dict[str, float],
) -> Dict[str, float]:
    """
    Compute all module-4 intracellular fluxes.
    """
    fluxes = {
        "ISF_to_cell": params["k_ISF_to_cell"] * A_brain_ISF,
        "cell_to_nuc": params["k_cell_to_nuc"] * A_brain_cell,
        "cell_loss": params["k_cell_loss"] * A_brain_cell,
        "deg_v": params["k_deg_v"] * A_brain_nuc,
        "tx": params["k_tx"] * A_brain_nuc,
        "deg_m": params["k_deg_m"] * mRNA_brain,
        "tl": params["k_tl"] * mRNA_brain,
        "deg_p": params["k_deg_p"] * P_brain,
    }
    return fluxes


def aav_intracellular_rhs(
    t: float,
    y: np.ndarray,
    params: Dict[str, float],
    idx: Dict[str, int],
) -> np.ndarray:
    """
    Module 4: brain cell uptake and expression.
```





```
    States affected:
    - A_brain_ISF
    - A_brain_cell
    - A_brain_nuc
    - mRNA_brain
    - P_brain
    - A_cleared (for cell_loss + deg_v + deg_m + deg_p if you want to accumulate losses)
    """
    _ = t
    validate_intracellular_params(params)

    dydt = np.zeros_like(y)

    A_brain_ISF = y[idx["A_brain_ISF"]]
    A_brain_cell = y[idx["A_brain_cell"]]
    A_brain_nuc = y[idx["A_brain_nuc"]]
    mRNA_brain = y[idx["mRNA_brain"]]
    P_brain = y[idx["P_brain"]]

    fluxes = compute_intracellular_fluxes(
        A_brain_ISF=A_brain_ISF,
        A_brain_cell=A_brain_cell,
        A_brain_nuc=A_brain_nuc,
        mRNA_brain=mRNA_brain,
        P_brain=P_brain,
        params=params,
    )

    # A_brain_ISF:
    # lose by cell uptake
    dydt[idx["A_brain_ISF"]] += -fluxes["ISF_to_cell"]

    # A_brain_cell:
    # gain from ISF, lose to nucleus and cell-processing loss
    dydt[idx["A_brain_cell"]] += (
        fluxes["ISF_to_cell"] - fluxes["cell_to_nuc"] - fluxes["cell_loss"]
    )

    # A_brain_nuc:
    # gain from cell, lose by degradation
    dydt[idx["A_brain_nuc"]] += fluxes["cell_to_nuc"] - fluxes["deg_v"]

    # mRNA_brain:
    # gain by transcription, lose by degradation
    dydt[idx["mRNA_brain"]] += fluxes["tx"] - fluxes["deg_m"]

    # P_brain:
    # gain by translation, lose by degradation
    dydt[idx["P_brain"]] += fluxes["tl"] - fluxes["deg_p"]

    # Optional accumulation into cleared pool
    dydt[idx["A_cleared"]] += (
        fluxes["cell_loss"] + fluxes["deg_v"] + fluxes["deg_m"] + fluxes["deg_p"]
    )

    return dydt


def extract_intracellular_flux_trajectories(
    t_array: np.ndarray,
```





```
    y_array: np.ndarray,
    params: Dict[str, float],
    idx: Dict[str, int],
) -> Dict[str, np.ndarray]:
    """
    Reconstruct module-4 intracellular flux trajectories from simulation result.
    """
    flux_history = {
        "ISF_to_cell": [],
        "cell_to_nuc": [],
        "cell_loss": [],
        "deg_v": [],
        "tx": [],
        "deg_m": [],
        "tl": [],
        "deg_p": [],
    }

    for k in range(y_array.shape[1]):
        A_brain_ISF = y_array[idx["A_brain_ISF"], k]
        A_brain_cell = y_array[idx["A_brain_cell"], k]
        A_brain_nuc = y_array[idx["A_brain_nuc"], k]
        mRNA_brain = y_array[idx["mRNA_brain"], k]
        P_brain = y_array[idx["P_brain"], k]

        fluxes = compute_intracellular_fluxes(
            A_brain_ISF=A_brain_ISF,
            A_brain_cell=A_brain_cell,
            A_brain_nuc=A_brain_nuc,
            mRNA_brain=mRNA_brain,
            P_brain=P_brain,
            params=params,
        )

        for name in flux_history:
            flux_history[name].append(fluxes[name])

    for name in flux_history:
        flux_history[name] = np.asarray(flux_history[name], dtype=float)

    return flux_history
```





models/editing/apoe_multisite_editing.py

```python
from __future__ import annotations

from typing import Dict
import numpy as np


REQUIRED_KEYS = (
    "editor_type",
    "k_on_112",
    "k_off_112",
    "k_cat_112",
    "k_on_158",
    "k_off_158",
    "k_cat_158",
    "k_prod_apoe",
    "k_deg_apoe",
    "local_bystander_per_112",
    "local_bystander_per_158",
    "k_on_puf_off",
    "k_off_puf_off",
    "k_cat_puf_off",
    "k_deaminase_bg",
    "k_prod_puf_off",
    "k_deg_puf_off",
    "k_prod_deaminase_bg",
    "k_deg_deaminase_bg",
)


def validate_params(params: Dict) -> None:
    for key in REQUIRED_KEYS:
        if key not in params:
            raise KeyError(f"Missing APOE multisite editing parameter: {key}")
        if key != "editor_type" and float(params[key]) < 0:
            raise ValueError(f"APOE multisite parameter '{key}' must be non-negative.")


def _positive(value: float) -> float:
    return max(float(value), 0.0)


def _scale_for_editor_type(params: Dict) -> Dict[str, float]:
    """
    Return editor-specific modifiers.

    v4 explicitly distinguishes APOBEC3A/A3A-like editors from APOBEC1-like
    editors. These are not final literature-calibrated constants; they are
    transparent priors that prevent us from mixing A3A and APOBEC1 mechanisms.
    """
    editor_type = str(params.get("editor_type", "A3A")).lower()
    if editor_type in {"a3a", "apobec3a", "engineered_a3a"}:
        return {
            "cat_scale": 1.0,
            "uc_context_scale": float(params.get("uc_context_scale", 1.2)),
            "background_scale": float(params.get("a3a_background_scale", 1.0)),
        }
```





```
    if editor_type in {"apobec1", "apobec1_like"}:
        return {
            "cat_scale": float(params.get("apobec1_cat_scale", 0.7)),
            "uc_context_scale": float(params.get("apobec1_uc_context_scale", 0.9)),
            "background_scale": float(params.get("apobec1_background_scale", 0.4)),
        }
    if editor_type in {"proapobec", "proapobec_like"}:
        return {
            "cat_scale": float(params.get("proapobec_cat_scale", 1.25)),
            "uc_context_scale": float(params.get("proapobec_uc_context_scale", 1.1)),
            "background_scale": float(params.get("proapobec_background_scale", 0.5)),
        }
    raise ValueError(f"Unsupported editor_type '{params.get('editor_type')}'.")


def compute_apoe_multisite_fluxes(
    y: np.ndarray, params: Dict, idx: Dict[str, int]
) -> Dict[str, float]:
    """
    Compute v4 APOE112/APOE158 and three-class off-target fluxes.

    APOE biology encoded here:
    - APOE4 has Arg112/Arg158.
    - Editing 112 only gives APOE3-like.
    - Editing both 112 and 158 gives APOE2-like.
    - Editing 158 only is tracked as an incomplete/mixed edit because it is not
      the normal APOE3-like therapeutic objective.
    """
    validate_params(params)
    editor_mod = _scale_for_editor_type(params)

    p = _positive(y[idx["P_brain"]])
    s4 = _positive(y[idx["S_APOE4"]])
    s3 = _positive(y[idx["S_APOE3_like"]])
    s2 = _positive(y[idx["S_APOE2_like"]])
    s158_only = _positive(y[idx["S_APOE158_only"]])
    c112 = _positive(y[idx["C_APOE112"]])
    c158 = _positive(y[idx["C_APOE158"]])
    c158_after112 = _positive(y[idx["C_APOE158_after112"]])
    c112_after158 = _positive(y[idx["C_APOE112_after158"]])
    s_puf_off = _positive(y[idx["S_puf_off"]])
    c_puf_off = _positive(y[idx["C_puf_off"]])
    s_deam = _positive(y[idx["S_deaminase_bg"]])

    cat112 = (
        params["k_cat_112"] * editor_mod["cat_scale"] * editor_mod["uc_context_scale"]
    )
    cat158 = params["k_cat_158"] * editor_mod["cat_scale"]
    cat_off = params["k_cat_puf_off"] * editor_mod["cat_scale"]
    k_bg = params["k_deaminase_bg"] * editor_mod["background_scale"]

    bind112 = params["k_on_112"] * p * s4
    unbind112 = params["k_off_112"] * c112
    edit112 = cat112 * c112

    bind158 = params["k_on_158"] * p * s4
    unbind158 = params["k_off_158"] * c158
    edit158 = cat158 * c158

    bind158_after112 = params["k_on_158"] * p * s3
    unbind158_after112 = params["k_off_158"] * c158_after112
```





```
    edit158_after112 = cat158 * c158_after112

    bind112_after158 = params["k_on_112"] * p * s158_only
    unbind112_after158 = params["k_off_112"] * c112_after158
    edit112_after158 = cat112 * c112_after158

    bind_puf_off = params["k_on_puf_off"] * p * s_puf_off
    unbind_puf_off = params["k_off_puf_off"] * c_puf_off
    edit_puf_off = cat_off * c_puf_off

    deaminase_bg_edit = k_bg * p * s_deam
    local_bystander = params["local_bystander_per_112"] * (
        edit112 + edit112_after158
    ) + params["local_bystander_per_158"] * (edit158 + edit158_after112)

    return {
        "prod_apoe": params["k_prod_apoe"],
        "deg_apoe4": params["k_deg_apoe"] * s4,
        "deg_apoe3_like": params["k_deg_apoe"] * s3,
        "deg_apoe2_like": params["k_deg_apoe"] * s2,
        "deg_apoe158_only": params["k_deg_apoe"] * s158_only,
        "bind112": bind112,
        "unbind112": unbind112,
        "edit112": edit112,
        "bind158": bind158,
        "unbind158": unbind158,
        "edit158": edit158,
        "bind158_after112": bind158_after112,
        "unbind158_after112": unbind158_after112,
        "edit158_after112": edit158_after112,
        "bind112_after158": bind112_after158,
        "unbind112_after158": unbind112_after158,
        "edit112_after158": edit112_after158,
        "local_bystander": local_bystander,
        "prod_puf_off": params["k_prod_puf_off"],
        "deg_puf_off": params["k_deg_puf_off"] * s_puf_off,
        "bind_puf_off": bind_puf_off,
        "unbind_puf_off": unbind_puf_off,
        "edit_puf_off": edit_puf_off,
        "prod_deaminase_bg": params["k_prod_deaminase_bg"],
        "deg_deaminase_bg": params["k_deg_deaminase_bg"] * s_deam,
        "edit_deaminase_bg": deaminase_bg_edit,
    }


def apoe_multisite_rhs(
    t: float, y: np.ndarray, params: Dict, idx: Dict[str, int]
) -> np.ndarray:
    _ = t
    fluxes = compute_apoe_multisite_fluxes(y, params, idx)
    dydt = np.zeros_like(y)

    editor_binding_loss = (
        fluxes["bind112"]
        + fluxes["bind158"]
        + fluxes["bind158_after112"]
        + fluxes["bind112_after158"]
        + fluxes["bind_puf_off"]
    )
    editor_release = (
        fluxes["unbind112"]
```





```
        + fluxes["unbind158"]
        + fluxes["unbind158_after112"]
        + fluxes["unbind112_after158"]
        + fluxes["unbind_puf_off"]
        + fluxes["edit112"]
        + fluxes["edit158"]
        + fluxes["edit158_after112"]
        + fluxes["edit112_after158"]
        + fluxes["edit_puf_off"]
    )
    dydt[idx["P_brain"]] += -editor_binding_loss + editor_release

    dydt[idx["S_APOE4"]] += (
        fluxes["prod_apoe"]
        - fluxes["deg_apoe4"]
        - fluxes["bind112"]
        + fluxes["unbind112"]
        - fluxes["bind158"]
        + fluxes["unbind158"]
    )
    dydt[idx["C_APOE112"]] += fluxes["bind112"] - fluxes["unbind112"] - fluxes["edit112"]
    dydt[idx["C_APOE158"]] += fluxes["bind158"] - fluxes["unbind158"] - fluxes["edit158"]

    dydt[idx["S_APOE3_like"]] += (
        fluxes["edit112"]
        - fluxes["deg_apoe3_like"]
        - fluxes["bind158_after112"]
        + fluxes["unbind158_after112"]
    )
    dydt[idx["C_APOE158_after112"]] += (
        fluxes["bind158_after112"]
        - fluxes["unbind158_after112"]
        - fluxes["edit158_after112"]
    )

    dydt[idx["S_APOE158_only"]] += (
        fluxes["edit158"]
        - fluxes["deg_apoe158_only"]
        - fluxes["bind112_after158"]
        + fluxes["unbind112_after158"]
    )
    dydt[idx["C_APOE112_after158"]] += (
        fluxes["bind112_after158"]
        - fluxes["unbind112_after158"]
        - fluxes["edit112_after158"]
    )

    dydt[idx["S_APOE2_like"]] += (
        fluxes["edit158_after112"] + fluxes["edit112_after158"] - fluxes["deg_apoe2_like"]
    )

    dydt[idx["B_local_bystander"]] += fluxes["local_bystander"]

    dydt[idx["S_puf_off"]] += (
        fluxes["prod_puf_off"]
        - fluxes["deg_puf_off"]
        - fluxes["bind_puf_off"]
        + fluxes["unbind_puf_off"]
    )
    dydt[idx["C_puf_off"]] += (
        fluxes["bind_puf_off"] - fluxes["unbind_puf_off"] - fluxes["edit_puf_off"]
```





```
    )
    dydt[idx["E_puf_off"]] += fluxes["edit_puf_off"]

    dydt[idx["S_deaminase_bg"]] += (
        fluxes["prod_deaminase_bg"]
        - fluxes["deg_deaminase_bg"]
        - fluxes["edit_deaminase_bg"]
    )
    dydt[idx["E_deaminase_bg"]] += fluxes["edit_deaminase_bg"]

    # Compatibility accumulators used by older optimization/reporting code.
    dydt[idx["E_on"]] += fluxes["edit112"] + fluxes["edit158_after112"]
    dydt[idx["E_off"]] += (
        fluxes["local_bystander"] + fluxes["edit_puf_off"] + fluxes["edit_deaminase_bg"]
    )

    return dydt


def extract_apoe_multisite_flux_trajectories(
    t_array: np.ndarray,
    y_array: np.ndarray,
    params: Dict,
    idx: Dict[str, int],
) -> Dict[str, np.ndarray]:
    _ = t_array
    history: Dict[str, list[float]] = {}
    for col in range(y_array.shape[1]):
        fluxes = compute_apoe_multisite_fluxes(y_array[:, col], params, idx)
        for key, value in fluxes.items():
            history.setdefault(key, []).append(value)
    return {key: np.asarray(values, dtype=float) for key, values in history.items()}


def compute_apoe_multisite_metrics(
    y: np.ndarray, idx: Dict[str, int], config: Dict
) -> Dict[str, np.ndarray]:
    eps = float(config["editing"].get("specificity_eps", 1e-12))
    apoe4 = y[idx["S_APOE4"], :]
    apoe3 = y[idx["S_APOE3_like"], :]
    apoe2 = y[idx["S_APOE2_like"], :]
    apoe158_only = y[idx["S_APOE158_only"], :]
    complexes = (
        y[idx["C_APOE112"], :]
        + y[idx["C_APOE158"], :]
        + y[idx["C_APOE158_after112"], :]
        + y[idx["C_APOE112_after158"], :]
    )
    total_apoe = apoe4 + apoe3 + apoe2 + apoe158_only + complexes + eps

    local = y[idx["B_local_bystander"], :]
    puf_off = y[idx["E_puf_off"], :]
    deam = y[idx["E_deaminase_bg"], :]
    off_total = local + puf_off + deam

    therapeutic = apoe3 + apoe2
    apoe3_like_fraction = apoe3 / total_apoe
    apoe2_like_fraction = apoe2 / total_apoe
    apoe4_fraction = apoe4 / total_apoe
    mixed_edit_fraction = apoe158_only / total_apoe

```





```
    return {
        "on_target_editing_rate": therapeutic / total_apoe,
        "off_target_burden": off_total / total_apoe,
        "specificity_index": therapeutic / (off_total + eps),
        "on_target_editing_fraction": therapeutic / total_apoe,
        "off_target_editing_fraction": off_total / (total_apoe + off_total + eps),
        "apoe4_fraction": apoe4_fraction,
        "apoe3_like_fraction": apoe3_like_fraction,
        "apoe2_like_fraction": apoe2_like_fraction,
        "mixed_edit_fraction": mixed_edit_fraction,
        "local_bystander_burden": local / total_apoe,
        "puf_mismatch_burden": puf_off / total_apoe,
        "deaminase_background_burden": deam / total_apoe,
        "ldlr_binding_risk_proxy": apoe2 / total_apoe,
    }
```





models/editing/competitive_editing.py

```python
from __future__ import annotations

from typing import Dict
import numpy as np


def validate_editing_params(params: Dict[str, float]) -> None:
    required = [
        "k_cat_on",
        "K_d_on",
        "k_cat_off",
        "K_d_off",
        "k_loss_on",
        "k_loss_off",
    ]
    for key in required:
        if key not in params:
            raise KeyError(f"Missing editing parameter: {key}")
        if params[key] < 0:
            raise ValueError(f"Editing parameter '{key}' must be non-negative.")


def compute_editing_fluxes(
    P_brain: float,
    S_on: float,
    S_off: float,
    E_on: float,
    E_off: float,
    params: Dict[str, float],
) -> Dict[str, float]:
    """
    Competitive editing fluxes for module 5.
    """
    k_cat_on = params["k_cat_on"]
    K_d_on = params["K_d_on"]
    k_cat_off = params["k_cat_off"]
    K_d_off = params["K_d_off"]
    k_loss_on = params["k_loss_on"]
    k_loss_off = params["k_loss_off"]

    # Competitive catalytic forms
    v_on = (
        k_cat_on
        * P_brain
        * S_on
        / (K_d_on * (1.0 + S_off / max(K_d_off, 1e-12)) + S_on + 1e-12)
    )

    v_off = (
        k_cat_off
        * P_brain
        * S_off
        / (K_d_off * (1.0 + S_on / max(K_d_on, 1e-12)) + S_off + 1e-12)
    )

    fluxes = {
```





```
        "v_on": v_on,
        "v_off": v_off,
        "loss_on": k_loss_on * E_on,
        "loss_off": k_loss_off * E_off,
    }
    return fluxes


def competitive_editing_rhs(
    t: float,
    y: np.ndarray,
    params: Dict[str, float],
    idx: Dict[str, int],
) -> np.ndarray:
    """
    Module 5: competitive on-target / off-target editing dynamics.

    States affected:
    - S_on
    - S_off
    - E_on
    - E_off
    """
    _ = t
    validate_editing_params(params)

    dydt = np.zeros_like(y)

    P_brain = y[idx["P_brain"]]
    S_on = y[idx["S_on"]]
    S_off = y[idx["S_off"]]
    E_on = y[idx["E_on"]]
    E_off = y[idx["E_off"]]

    fluxes = compute_editing_fluxes(
        P_brain=P_brain,
        S_on=S_on,
        S_off=S_off,
        E_on=E_on,
        E_off=E_off,
        params=params,
    )

    v_on = fluxes["v_on"]
    v_off = fluxes["v_off"]
    loss_on = fluxes["loss_on"]
    loss_off = fluxes["loss_off"]

    # substrate depletion
    dydt[idx["S_on"]] += -v_on
    dydt[idx["S_off"]] += -v_off

    # edited outcome accumulation
    dydt[idx["E_on"]] += v_on - loss_on
    dydt[idx["E_off"]] += v_off - loss_off

    return dydt


def extract_editing_flux_trajectories(
    t_array: np.ndarray,
```





```
    y_array: np.ndarray,
    params: Dict[str, float],
    idx: Dict[str, int],
) -> Dict[str, np.ndarray]:
    """
    Reconstruct module-5 editing flux trajectories from simulation result.
    """
    flux_history = {
        "v_on": [],
        "v_off": [],
        "loss_on": [],
        "loss_off": [],
    }

    for k in range(y_array.shape[1]):
        P_brain = y_array[idx["P_brain"], k]
        S_on = y_array[idx["S_on"], k]
        S_off = y_array[idx["S_off"], k]
        E_on = y_array[idx["E_on"], k]
        E_off = y_array[idx["E_off"], k]

        fluxes = compute_editing_fluxes(
            P_brain=P_brain,
            S_on=S_on,
            S_off=S_off,
            E_on=E_on,
            E_off=E_off,
            params=params,
        )

        for name in flux_history:
            flux_history[name].append(fluxes[name])

    for name in flux_history:
        flux_history[name] = np.asarray(flux_history[name], dtype=float)

    return flux_history


def compute_editing_metrics(
    y: np.ndarray,
    idx: Dict[str, int],
    config: Dict,
) -> Dict[str, np.ndarray]:
    """
    Compute derived module-5 metrics:
    - on-target editing rate
    - off-target burden
    - specificity index
    """
    S_on_init = float(config["editing"]["S_on_init"])
    S_off_init = float(config["editing"]["S_off_init"])
    eps = float(config["editing"].get("specificity_eps", 1e-12))

    E_on = y[idx["E_on"], :]
    E_off = y[idx["E_off"], :]

    on_target_editing_rate = E_on / max(S_on_init, eps)
```





```
    off_target_burden = E_off / max(S_off_init, eps)
    specificity_index = E_on / (E_off + eps)

    return {
        "on_target_editing_rate": on_target_editing_rate,
        "off_target_burden": off_target_burden,
        "specificity_index": specificity_index,
    }
```





models/editing/module5.py

```python
from __future__ import annotations

from typing import Dict
import numpy as np

from models.editing.competitive_editing import (
    competitive_editing_rhs,
    extract_editing_flux_trajectories as extract_competitive_flux_trajectories,
    compute_editing_metrics as compute_competitive_metrics,
)
from models.editing.puf_apobec_mechanistic import (
    puf_apobec_mechanistic_rhs,
    extract_mechanistic_flux_trajectories,
    compute_mechanistic_editing_metrics,
)
from models.editing.apoe_multisite_editing import (
    apoe_multisite_rhs,
    extract_apoe_multisite_flux_trajectories,
    compute_apoe_multisite_metrics,
)


def get_editing_model(params: Dict) -> str:
    """
    Return the selected module-5 model.

    Supported values:
    - competitive: original simplified competitive editing model
    - mechanistic_puf_apobec: explicit binding/unbinding/catalysis model
    - apoe_multisite: APOE112/APOE158-aware v4 model
    """
    return str(params.get("model", "competitive"))


def editing_rhs(t: float, y: np.ndarray, params: Dict, idx: Dict[str, int]) -> np.ndarray:
    model = get_editing_model(params)
    if model == "competitive":
        return competitive_editing_rhs(t, y, params, idx)
    if model == "mechanistic_puf_apobec":
        return puf_apobec_mechanistic_rhs(t, y, params, idx)
    if model == "apoe_multisite":
        return apoe_multisite_rhs(t, y, params, idx)
    raise ValueError(f"Unknown editing model '{model}'.")


def extract_editing_flux_trajectories(
    t_array: np.ndarray,
    y_array: np.ndarray,
    params: Dict,
    idx: Dict[str, int],
) -> Dict[str, np.ndarray]:
    model = get_editing_model(params)
    if model == "competitive":
        return extract_competitive_flux_trajectories(t_array, y_array, params, idx)
    if model == "mechanistic_puf_apobec":
        return extract_mechanistic_flux_trajectories(t_array, y_array, params, idx)
```





```
    if model == "apoe_multisite":
        return extract_apoe_multisite_flux_trajectories(t_array, y_array, params, idx)
    raise ValueError(f"Unknown editing model '{model}'.")


def compute_editing_metrics(
    y: np.ndarray, idx: Dict[str, int], config: Dict
) -> Dict[str, np.ndarray]:
    model = get_editing_model(config["editing"])
    if model == "competitive":
        return compute_competitive_metrics(y, idx, config)
    if model == "mechanistic_puf_apobec":
        return compute_mechanistic_editing_metrics(y, idx, config)
    if model == "apoe_multisite":
        return compute_apoe_multisite_metrics(y, idx, config)
    raise ValueError(f"Unknown editing model '{model}'.")
```





models/editing/offtarget_panel.py

```python
from __future__ import annotations

from copy import deepcopy
from pathlib import Path
from typing import Dict, Iterable, List
import csv


def load_offtarget_panel(path: str | Path) -> List[Dict[str, float | str]]:
    """
    Load a REWIRE-style off-target candidate panel.

    Each row represents one candidate RNA site. The model does not yet track
    every site as a separate ODE state, so this module compresses the panel into
    an effective aggregate off-target pool. That is a pragmatic bridge between
    transcriptome-wide dry-lab screening and the current compact ODE system.
    """
    rows: List[Dict[str, float | str]] = []
    with open(path, "r", encoding="utf-8", newline="") as f:
        reader = csv.DictReader(f)
        for raw in reader:
            rows.append(
                {
                    "site_id": raw["site_id"],
                    "gene": raw.get("gene", ""),
                    "initial_pool": float(raw.get("initial_pool") or 1.0),
                    "binding_score": float(raw.get("binding_score") or 0.0),
                    "accessibility": float(raw.get("accessibility") or 1.0),
                    "context_score": float(raw.get("context_score") or 1.0),
                    "validation_priority": raw.get("validation_priority", "medium"),
                }
            )
    return rows


def summarize_offtarget_panel(rows: Iterable[Dict[str, float | str]]) -> Dict[str, float]:
    """
    Convert candidate-site rows to effective aggregate model quantities.

    Interpretation:
    - `effective_pool` becomes `editing.S_off_init`.
    - `relative_binding` scales `editing.k_on_off`.
    - `relative_catalysis` scales `editing.k_cat_off`.

    The weights intentionally use accessibility because hidden RNA sites should
    contribute less to binding and editing burden.
    """
    rows = list(rows)
    if not rows:
        return {
            "n_sites": 0,
            "effective_pool": 0.0,
            "relative_binding": 0.0,
            "relative_catalysis": 0.0,
        }


```





```
    weighted_pool = 0.0
    weighted_binding = 0.0
    weighted_catalysis = 0.0

    for row in rows:
        pool = float(row["initial_pool"])
        accessibility = max(float(row["accessibility"]), 0.0)
        binding_score = max(float(row["binding_score"]), 0.0)
        context_score = max(float(row["context_score"]), 0.0)
        weight = pool * accessibility

        weighted_pool += weight
        weighted_binding += weight * binding_score
        weighted_catalysis += weight * binding_score * context_score

    if weighted_pool <= 0:
        relative_binding = 0.0
        relative_catalysis = 0.0
    else:
        relative_binding = weighted_binding / weighted_pool
        relative_catalysis = weighted_catalysis / weighted_pool

    return {
        "n_sites": float(len(rows)),
        "effective_pool": weighted_pool,
        "relative_binding": relative_binding,
        "relative_catalysis": relative_catalysis,
    }


def apply_offtarget_panel_to_config(
    config: Dict,
    panel_path: str | Path,
    min_pool: float = 1e-9,
) -> tuple[Dict, Dict[str, float]]:
    """
    Return a config copy with aggregate off-target parameters updated.

    This keeps the current ODE model compact while letting the off-target risk
    come from a transparent candidate table instead of a hand-picked `S_off`.
    """
    rows = load_offtarget_panel(panel_path)
    return apply_offtarget_rows_to_config(config, rows, min_pool=min_pool)


def apply_offtarget_rows_to_config(
    config: Dict,
    rows: Iterable[Dict[str, float | str]],
    min_pool: float = 1e-9,
) -> tuple[Dict, Dict[str, float]]:
    """Apply already-validated candidate rows without requiring a temporary file."""
    updated = deepcopy(config)
    summary = summarize_offtarget_panel(rows)
    editing = updated["editing"]

    if editing.get("model") == "apoe_multisite":
        pool_key = "S_puf_off_init"
        kon_key = "k_on_puf_off"
        kcat_key = "k_cat_puf_off"
    else:
        pool_key = "S_off_init"
```





```
        kon_key = "k_on_off"
        kcat_key = "k_cat_off"

    base_pool = max(float(editing.get(pool_key, 1.0)), min_pool)
    base_k_on = float(editing.get(kon_key, 0.0))
    base_k_cat = float(editing.get(kcat_key, 0.0))

    effective_pool = max(summary["effective_pool"], min_pool)
    editing[pool_key] = effective_pool

    # Keep the original parameter values as priors, then scale by panel-derived
    # relative risk. The clamp avoids accidentally erasing off-target risk when
    # an early placeholder panel has sparse scores.
    editing[kon_key] = base_k_on * max(summary["relative_binding"], 0.05)
    editing[kcat_key] = base_k_cat * max(summary["relative_catalysis"], 0.05)

    summary["pool_key"] = pool_key
    summary["base_offtarget_pool"] = base_pool
    summary["applied_offtarget_pool"] = editing[pool_key]
    summary["applied_offtarget_k_on"] = editing[kon_key]
    summary["applied_offtarget_k_cat"] = editing[kcat_key]

    return updated, summary
```





models/editing/puf_apobec_mechanistic.py

```python
from __future__ import annotations

from typing import Dict
import numpy as np


REQUIRED_MECHANISTIC_KEYS = (
    "k_on_on",
    "k_off_on",
    "k_cat_on",
    "k_on_off",
    "k_off_off",
    "k_cat_off",
    "k_prod_on",
    "k_deg_on",
    "k_prod_off",
    "k_deg_off",
    "k_loss_on",
    "k_loss_off",
)


def validate_mechanistic_params(params: Dict[str, float]) -> None:
    """
    Validate the explicit PUF-APOBEC editing parameters.

    This module combines two ideas:
    - The GitHub model supplies P_brain, the active editor made after AAV
      delivery, BBB transport, cell uptake, transcription, and translation.
    - The local notebook describes enzyme kinetics: E + S <-> ES -> E + P.

    Here P_brain is treated as the free active editor E. It can bind target
    APOE4 RNA or off-target RNA, form enzyme-substrate complexes, catalyze
    editing, and then be released for reuse.
    """
    for key in REQUIRED_MECHANISTIC_KEYS:
        if key not in params:
            raise KeyError(f"Missing mechanistic editing parameter: {key}")
        if params[key] < 0:
            raise ValueError(
                f"Mechanistic editing parameter '{key}' must be non-negative."
            )


def compute_mechanistic_editing_fluxes(
    P_brain: float,
    S_on: float,
    S_off: float,
    ES_on: float,
    ES_off: float,
    E_on: float,
    E_off: float,
    params: Dict[str, float],
) -> Dict[str, float]:
    """
    Compute all reaction fluxes for the combined PUF-APOBEC module.
```





```

    Naming convention
    -----------------
    P_brain:
        Free active PUF-APOBEC editor in the brain cell compartment. This is
        produced by module 4 and plays the role of E in the notebook.
    S_on / S_off:
        Unedited on-target APOE4 RNA and accessible off-target RNA pools.
    ES_on / ES_off:
        Explicit enzyme-substrate complexes for the two RNA pools.
    E_on / E_off:
        Edited products accumulated from on-target and off-target catalysis.

    The two substrates compete because both binding fluxes consume the same
    free editor pool P_brain.
    """
    validate_mechanistic_params(params)

    # Numerical guard: LSODA can briefly propose tiny negative values during
    # stiff transitions. Clipping only for flux calculation avoids artificial
    # negative reaction rates while leaving the solver state untouched.
    E_free = max(float(P_brain), 0.0)
    S_on_free = max(float(S_on), 0.0)
    S_off_free = max(float(S_off), 0.0)
    C_on = max(float(ES_on), 0.0)
    C_off = max(float(ES_off), 0.0)

    bind_on = params["k_on_on"] * E_free * S_on_free
    unbind_on = params["k_off_on"] * C_on
    edit_on = params["k_cat_on"] * C_on

    bind_off = params["k_on_off"] * E_free * S_off_free
    unbind_off = params["k_off_off"] * C_off
    edit_off = params["k_cat_off"] * C_off

    return {
        "prod_on": params["k_prod_on"],
        "deg_on": params["k_deg_on"] * S_on_free,
        "bind_on": bind_on,
        "unbind_on": unbind_on,
        "edit_on": edit_on,
        "prod_off": params["k_prod_off"],
        "deg_off": params["k_deg_off"] * S_off_free,
        "bind_off": bind_off,
        "unbind_off": unbind_off,
        "edit_off": edit_off,
        "loss_on": params["k_loss_on"] * max(float(E_on), 0.0),
        "loss_off": params["k_loss_off"] * max(float(E_off), 0.0),
    }


def puf_apobec_mechanistic_rhs(
    t: float,
    y: np.ndarray,
    params: Dict[str, float],
    idx: Dict[str, int],
) -> np.ndarray:
    """
    RHS for the mechanistic PUF-APOBEC editing module.

    Compared with the original competitive Michaelis-Menten style module, this
```





```
    explicitly tracks transient complexes:

        P_brain + S_on  <->  ES_on   ->  P_brain + E_on
        P_brain + S_off <->  ES_off  ->  P_brain + E_off

    P_brain is also changed by module 4 in the same global RHS. The updates
    below only add the binding/catalysis contribution to the editor pool.
    """
    _ = t
    validate_mechanistic_params(params)

    dydt = np.zeros_like(y)

    P_brain = y[idx["P_brain"]]
    S_on = y[idx["S_on"]]
    S_off = y[idx["S_off"]]
    ES_on = y[idx["ES_on"]]
    ES_off = y[idx["ES_off"]]
    E_on = y[idx["E_on"]]
    E_off = y[idx["E_off"]]

    fluxes = compute_mechanistic_editing_fluxes(
        P_brain=P_brain,
        S_on=S_on,
        S_off=S_off,
        ES_on=ES_on,
        ES_off=ES_off,
        E_on=E_on,
        E_off=E_off,
        params=params,
    )

    # Free active editor. Binding temporarily removes editor from the free
    # pool; unbinding and catalysis release it back, matching the notebook's
    # enzyme recycling term.
    dydt[idx["P_brain"]] += (
        -fluxes["bind_on"]
        - fluxes["bind_off"]
        + fluxes["unbind_on"]
        + fluxes["unbind_off"]
        + fluxes["edit_on"]
        + fluxes["edit_off"]
    )

    # On-target APOE4 RNA: renewed by transcription/turnover, naturally
    # degraded, consumed by binding, and restored if the complex dissociates.
    dydt[idx["S_on"]] += (
        fluxes["prod_on"] - fluxes["deg_on"] - fluxes["bind_on"] + fluxes["unbind_on"]
    )

    # Off-target accessible RNA pool follows the same binding logic.
    dydt[idx["S_off"]] += (
        fluxes["prod_off"] - fluxes["deg_off"] - fluxes["bind_off"] + fluxes["unbind_off"]
    )

    # Enzyme-substrate complexes are formed by binding, then lost by either
    # dissociation or catalytic editing.
    dydt[idx["ES_on"]] += fluxes["bind_on"] - fluxes["unbind_on"] - fluxes["edit_on"]
    dydt[idx["ES_off"]] += fluxes["bind_off"] - fluxes["unbind_off"] - fluxes["edit_off"]

    # Edited product pools. E_on is the desired APOE2-like edited RNA output;
```





```
    # E_off is the safety burden from off-target edits.
    dydt[idx["E_on"]] += fluxes["edit_on"] - fluxes["loss_on"]
    dydt[idx["E_off"]] += fluxes["edit_off"] - fluxes["loss_off"]

    return dydt


def extract_mechanistic_flux_trajectories(
    t_array: np.ndarray,
    y_array: np.ndarray,
    params: Dict[str, float],
    idx: Dict[str, int],
) -> Dict[str, np.ndarray]:
    """
    Reconstruct mechanistic editing fluxes for plotting and diagnostics.
    """
    _ = t_array
    flux_history = {
        "prod_on": [],
        "deg_on": [],
        "bind_on": [],
        "unbind_on": [],
        "edit_on": [],
        "prod_off": [],
        "deg_off": [],
        "bind_off": [],
        "unbind_off": [],
        "edit_off": [],
        "loss_on": [],
        "loss_off": [],
    }

    for k in range(y_array.shape[1]):
        fluxes = compute_mechanistic_editing_fluxes(
            P_brain=y_array[idx["P_brain"], k],
            S_on=y_array[idx["S_on"], k],
            S_off=y_array[idx["S_off"], k],
            ES_on=y_array[idx["ES_on"], k],
            ES_off=y_array[idx["ES_off"], k],
            E_on=y_array[idx["E_on"], k],
            E_off=y_array[idx["E_off"], k],
            params=params,
        )
        for name in flux_history:
            flux_history[name].append(fluxes[name])

    for name in flux_history:
        flux_history[name] = np.asarray(flux_history[name], dtype=float)

    return flux_history


def compute_mechanistic_editing_metrics(
    y: np.ndarray,
    idx: Dict[str, int],
    config: Dict,
) -> Dict[str, np.ndarray]:
    """
    Compute observable editing metrics.

    The main editing fraction uses edited / (unedited + edited), which matches
```





```
    the local notebook. The initial-pool normalized metrics are kept for module
    6 dose optimization compatibility.
    """
    editing_cfg = config["editing"]
    eps = float(editing_cfg.get("specificity_eps", 1e-12))

    S_on = y[idx["S_on"], :]
    S_off = y[idx["S_off"], :]
    E_on = y[idx["E_on"], :]
    E_off = y[idx["E_off"], :]

    S_on_init = float(editing_cfg.get("S_on_init", max(float(S_on[0]), eps)))
    S_off_init = float(editing_cfg.get("S_off_init", max(float(S_off[0]), eps)))

    on_target_editing_fraction = E_on / (S_on + E_on + eps)
    off_target_editing_fraction = E_off / (S_off + E_off + eps)

    return {
        "on_target_editing_rate": E_on / max(S_on_init, eps),
        "off_target_burden": E_off / max(S_off_init, eps),
        "specificity_index": E_on / (E_off + eps),
        "on_target_editing_fraction": on_target_editing_fraction,
        "off_target_editing_fraction": off_target_editing_fraction,
    }
```





models/editing/sequence_to_kinetics.py

```python
from __future__ import annotations

from copy import deepcopy
from pathlib import Path
from typing import Dict, Iterable, List
import csv
import math


def sigmoid(x: float) -> float:
    return 1.0 / (1.0 + math.exp(-x))


def load_design_table(path: str | Path) -> List[Dict[str, float | str]]:
    """
    Load PUF-deaminase design candidates.

    The table is intentionally lightweight: it can be filled from hand-curated
    APOE target designs now and replaced by REWIRE/ProAPOBEC benchmark data
    later.
    """
    rows: List[Dict[str, float | str]] = []
    with open(path, "r", encoding="utf-8", newline="") as f:
        reader = csv.DictReader(f)
        for raw in reader:
            rows.append(
                {
                    "design_id": raw["design_id"],
                    "editor_type": raw.get("editor_type", "A3A"),
                    "puf_repeats": float(raw.get("puf_repeats") or 10),
                    "target_site": raw.get("target_site", "APOE112"),
                    "puf_score_112": float(raw.get("puf_score_112") or 0.0),
                    "puf_score_158": float(raw.get("puf_score_158") or 0.0),
                    "mismatch_count": float(raw.get("mismatch_count") or 0.0),
                    "accessibility_112": float(raw.get("accessibility_112") or 0.5),
                    "accessibility_158": float(raw.get("accessibility_158") or 0.5),
                    "distance_112": float(raw.get("distance_112") or 2.0),
                    "distance_158": float(raw.get("distance_158") or 2.0),
                    "uc_context_112": float(raw.get("uc_context_112") or 1.0),
                    "uc_context_158": float(raw.get("uc_context_158") or 0.5),
                    "local_bystander_risk": float(
                        raw.get("local_bystander_risk") or 0.05
                    ),
                    "deaminase_background_prior": float(
                        raw.get("deaminase_background_prior") or 1.0
                    ),
                    "editor_activity_scale": float(
                        raw.get("editor_activity_scale") or 1.0
                    ),
                }
            )
    return rows


def design_to_kinetic_modifiers(row: Dict[str, float | str]) -> Dict[str, float | str]:
    """
```





```
    Map sequence/design features to kinetic modifiers.

    This implements the optimization-plan idea:
    sequence/accessibility -> k_on, context/distance -> k_cat.
    """
    puf_repeats = float(row["puf_repeats"])
    mismatch_count = float(row["mismatch_count"])
    repeat_bonus = 1.0 + 0.10 * max(puf_repeats - 8.0, 0.0)
    mismatch_penalty = math.exp(-0.35 * mismatch_count)
    # Han et al. NAR 2022 reported a strong specificity gain when moving from
    # 8-repeat to 10-repeat PUF recognition. We encode that as a separate
    # off-target-risk modifier instead of only increasing on-target binding.
    puf_offtarget_scale = math.exp(-1.4 * max(puf_repeats - 8.0, 0.0)) * math.exp(
        0.45 * mismatch_count
    )

    kon112_scale = (
        math.exp(
            0.9 * float(row["puf_score_112"]) + 0.7 * float(row["accessibility_112"])
        )
        * repeat_bonus
        * mismatch_penalty
    )
    kon158_scale = (
        math.exp(
            0.9 * float(row["puf_score_158"]) + 0.7 * float(row["accessibility_158"])
        )
        * repeat_bonus
        * mismatch_penalty
    )

    distance112 = abs(float(row["distance_112"]) - 2.0)
    distance158 = abs(float(row["distance_158"]) - 2.0)
    # CU-REWIRE has a narrow C-to-U window, strongest around the second
    # nucleotide after the PUF binding site, and a strong UC-context preference.
    # The template stores context as a 0-1 relative motif score.
    kcat112_scale = (0.04 + 0.96 * float(row["uc_context_112"])) * math.exp(
        -0.9 * distance112
    )
    kcat158_scale = (0.04 + 0.96 * float(row["uc_context_158"])) * math.exp(
        -0.9 * distance158
    )
    editor_activity_scale = max(float(row.get("editor_activity_scale", 1.0)), 0.0)

    return {
        "design_id": row["design_id"],
        "editor_type": row["editor_type"],
        "kon112_scale": kon112_scale,
        "kon158_scale": kon158_scale,
        "kcat112_scale": kcat112_scale * editor_activity_scale,
        "kcat158_scale": kcat158_scale * editor_activity_scale,
        "local_bystander_scale": max(float(row["local_bystander_risk"]), 0.0),
        "deaminase_background_scale": max(float(row["deaminase_background_prior"]), 0.0)
        * editor_activity_scale,
        "editor_activity_scale": editor_activity_scale,
        "puf_offtarget_scale": max(puf_offtarget_scale, 0.0),
    }


def apply_design_to_config(
    config: Dict, row: Dict[str, float | str]
```





```
) -> tuple[Dict, Dict[str, float | str]]:
    updated = deepcopy(config)
    modifiers = design_to_kinetic_modifiers(row)
    editing = updated["editing"]

    editing["editor_type"] = str(modifiers["editor_type"])
    editing["k_on_112"] = float(editing["k_on_112"]) * float(modifiers["kon112_scale"])
    editing["k_on_158"] = float(editing["k_on_158"]) * float(modifiers["kon158_scale"])
    editing["k_cat_112"] = float(editing["k_cat_112"]) * float(modifiers["kcat112_scale"])
    editing["k_cat_158"] = float(editing["k_cat_158"]) * float(modifiers["kcat158_scale"])
    editing["local_bystander_per_112"] = float(modifiers["local_bystander_scale"])
    editing["local_bystander_per_158"] = 1.5 * float(modifiers["local_bystander_scale"])
    editing["k_on_puf_off"] = float(editing["k_on_puf_off"]) * float(
        modifiers["puf_offtarget_scale"]
    )
    editing["k_cat_puf_off"] = float(editing["k_cat_puf_off"]) * float(
        modifiers["puf_offtarget_scale"]
    )
    editing["k_deaminase_bg"] = float(editing["k_deaminase_bg"]) * float(
        modifiers["deaminase_background_scale"]
    )
    return updated, modifiers


def pareto_front(rows: Iterable[Dict[str, float]]) -> List[Dict[str, float]]:
    """
    Return nondominated design rows.

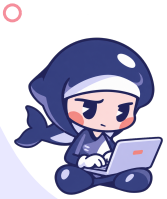
    We maximize utility, APOE3-like fraction and specificity; minimize
    off-target burden and LDLR-risk proxy.
    """
    rows = list(rows)
    front = []
    for row in rows:
        dominated = False
        for other in rows:
            if row is other:
                continue
            no_worse = (
                other["utility"] >= row["utility"]
                and other["apoe3_like_fraction_final"] >= row["apoe3_like_fraction_final"]
                and other["specificity_index_final"] >= row["specificity_index_final"]
                and other["off_target_burden_final"] <= row["off_target_burden_final"]
                and other["ldlr_binding_risk_proxy_final"]
                <= row["ldlr_binding_risk_proxy_final"]
            )
            strictly_better = (
                other["utility"] > row["utility"]
                or other["apoe3_like_fraction_final"] > row["apoe3_like_fraction_final"]
                or other["specificity_index_final"] > row["specificity_index_final"]
                or other["off_target_burden_final"] < row["off_target_burden_final"]
                or other["ldlr_binding_risk_proxy_final"]
                < row["ldlr_binding_risk_proxy_final"]
            )
            if no_worse and strictly_better:
                dominated = True
                break
        if not dominated:
            front.append(row)
    return front
```





models/calibration/__init__.py

```
"""Wet-lab calibration helpers for the iterated iGEM model."""
```





models/calibration/parameter_fit.py

```python
from __future__ import annotations

from copy import deepcopy
from pathlib import Path
from typing import Dict, Iterable, List
import csv
import math
import random

import yaml

from models.full_model.simulator import run_simulation
from models.calibration.wetlab_bridge import WetlabObservation, weighted_rmse
from optimization.iterated.uncertainty import sample_parameter


DEFAULT_FIT_PARAMETERS = [
    "distribution.k_blood_to_liver",
    "distribution.k_blood_to_brain",
    "intracellular.k_tx",
    "intracellular.k_tl",
    "intracellular.k_deg_p",
    "editing.k_cat_112",
    "editing.k_cat_158",
    "editing.k_on_puf_off",
    "editing.k_cat_puf_off",
    "editing.k_deaminase_bg",
]


def _set_nested(config: Dict, path: str, value: float) -> None:
    node = config
    parts = path.split(".")
    for part in parts[:-1]:
        node = node[part]
    node[parts[-1]] = float(value)


def _get_nested(config: Dict, path: str) -> float:
    node = config
    for part in path.split("."):
        node = node[part]
    return float(node)


def evaluate_config(
    config: Dict,
    observations: Iterable[WetlabObservation],
    dose: float,
    t_end: float,
    dt: float,
) -> tuple[float, Dict]:
    sim = run_simulation(config=config, dose=dose, t_end=t_end, dt=dt)
    return weighted_rmse(sim, observations), sim


```





```python
def random_search_fit(
    base_config: Dict,
    priors: Dict,
    observations: List[WetlabObservation],
    fit_parameters: List[str] | None = None,
    n_samples: int = 32,
    dose: float = 1.0,
    t_end: float = 72.0,
    dt: float = 0.2,
    seed: int = 11,
) -> tuple[Dict, List[Dict[str, float]]]:
    """
    Fit selected high-impact parameters by transparent random search.

    This is intentionally simple for iGEM use: it is easy to explain on a wiki,
    robust with sparse early wet-lab data, and can be replaced by scipy-based
    optimization once real replicate data arrive.
    """
    rng = random.Random(seed)
    fit_parameters = fit_parameters or DEFAULT_FIT_PARAMETERS
    fit_parameters = [p for p in fit_parameters if p in priors]

    baseline_score, _ = evaluate_config(base_config, observations, dose, t_end, dt)
    best_config = deepcopy(base_config)
    best_score = baseline_score
    rows: List[Dict[str, float]] = []

    baseline_row = {"sample_id": -1, "objective": baseline_score}
    for path in fit_parameters:
        baseline_row[path] = _get_nested(base_config, path)
    rows.append(baseline_row)

    for sample_id in range(int(n_samples)):
        config = deepcopy(base_config)
        row: Dict[str, float] = {"sample_id": float(sample_id)}
        for path in fit_parameters:
            value = sample_parameter(priors[path], rng)
            _set_nested(config, path, value)
            row[path] = value
        score, _ = evaluate_config(config, observations, dose, t_end, dt)
        if not math.isfinite(score):
            score = float("inf")
        row["objective"] = score
        rows.append(row)
        if score < best_score:
            best_score = score
            best_config = config

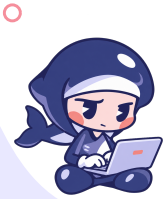
    return best_config, rows


def save_fit_results(rows: List[Dict[str, float]], path: str | Path) -> Path:
    path = Path(path)
    path.parent.mkdir(parents=True, exist_ok=True)
    fieldnames = sorted({key for row in rows for key in row.keys()})
    with open(path, "w", encoding="utf-8", newline="") as f:
        writer = csv.DictWriter(f, fieldnames=fieldnames)
        writer.writeheader()
        writer.writerows(rows)
    return path
```





```
117
118
119 def save_fit_config(config: Dict, path: str | Path) -> Path:
120     path = Path(path)
121     path.parent.mkdir(parents=True, exist_ok=True)
122     with open(path, "w", encoding="utf-8") as f:
123         yaml.safe_dump(config, f, sort_keys=False)
124     return path
```





models/calibration/wetlab_bridge.py

```python
from __future__ import annotations

from dataclasses import dataclass
from pathlib import Path
from typing import Dict, Iterable, List
import csv
import math

import numpy as np


@dataclass(frozen=True)
class WetlabObservation:
    """
    One measured wet-lab data point.

    The model itself works in arbitrary normalized units because the current
    project is still pre-calibration. This bridge lets us compare simulated
    trajectories to assay-level observations once the wet lab starts producing
    qPCR, amplicon sequencing, Western blot, or viability data.
    """

    assay: str
    time_h: float
    value: float
    sd: float = 1.0
    weight: float = 1.0


def _interp_state(results: Dict, state_name: str, time_h: float) -> float:
    """Interpolate one model state at the assay sampling time."""
    t = np.asarray(results["t"], dtype=float)
    y = np.asarray(results["y"], dtype=float)
    idx = results["idx"]
    return float(np.interp(float(time_h), t, y[idx[state_name], :]))


def simulated_assay_value(results: Dict, assay: str, time_h: float) -> float:
    """
    Convert model states into wet-lab observable quantities.

    Supported assays intentionally match realistic experiments a student team
    can run:
    - vector_qpcr_blood: vector genomes in blood by qPCR/ddPCR
    - vector_qpcr_liver: liver vector burden proxy
    - vector_qpcr_brain: brain/nuclear vector delivery proxy
    - puf_mrna_qpcr: expressed PUF-APOBEC mRNA
    - editor_western: active editor protein proxy
    - on_editing_amplicon_pct: on-target editing percent by amplicon sequencing
    - off_editing_amplicon_pct: off-target editing percent by amplicon sequencing
    - viability_pct: simple burden proxy, to be replaced by measured viability
    """
    eps = 1e-12
    assay = assay.strip()

    if assay == "vector_qpcr_blood":
```





```
        return _interp_state(results, "A_blood", time_h)
    if assay == "vector_qpcr_liver":
        return _interp_state(results, "A_liver", time_h)
    if assay == "vector_qpcr_brain":
        return _interp_state(results, "A_brain_nuc", time_h)
    if assay == "puf_mrna_qpcr":
        return _interp_state(results, "mRNA_brain", time_h)
    if assay == "editor_western":
        return _interp_state(results, "P_brain", time_h)

    if assay == "on_editing_amplicon_pct":
        if "S_APOE4" in results["idx"]:
            apoe4 = _interp_state(results, "S_APOE4", time_h)
            apoe3 = _interp_state(results, "S_APOE3_like", time_h)
            apoe2 = _interp_state(results, "S_APOE2_like", time_h)
            mixed = _interp_state(results, "S_APOE158_only", time_h)
            total = apoe4 + apoe3 + apoe2 + mixed + eps
            return 100.0 * (apoe3 + apoe2) / total
        s_on = _interp_state(results, "S_on", time_h)
        e_on = _interp_state(results, "E_on", time_h)
        return 100.0 * e_on / (s_on + e_on + eps)

    if assay == "apoe3_like_amplicon_pct":
        apoe4 = _interp_state(results, "S_APOE4", time_h)
        apoe3 = _interp_state(results, "S_APOE3_like", time_h)
        apoe2 = _interp_state(results, "S_APOE2_like", time_h)
        mixed = _interp_state(results, "S_APOE158_only", time_h)
        return 100.0 * apoe3 / (apoe4 + apoe3 + apoe2 + mixed + eps)

    if assay == "apoe2_like_amplicon_pct":
        apoe4 = _interp_state(results, "S_APOE4", time_h)
        apoe3 = _interp_state(results, "S_APOE3_like", time_h)
        apoe2 = _interp_state(results, "S_APOE2_like", time_h)
        mixed = _interp_state(results, "S_APOE158_only", time_h)
        return 100.0 * apoe2 / (apoe4 + apoe3 + apoe2 + mixed + eps)

    if assay == "off_editing_amplicon_pct":
        if "B_local_bystander" in results["idx"]:
            local = _interp_state(results, "B_local_bystander", time_h)
            puf_off = _interp_state(results, "E_puf_off", time_h)
            deam = _interp_state(results, "E_deaminase_bg", time_h)
            apoe4 = _interp_state(results, "S_APOE4", time_h)
            apoe3 = _interp_state(results, "S_APOE3_like", time_h)
            apoe2 = _interp_state(results, "S_APOE2_like", time_h)
            mixed = _interp_state(results, "S_APOE158_only", time_h)
            total = apoe4 + apoe3 + apoe2 + mixed + local + puf_off + deam + eps
            return 100.0 * (local + puf_off + deam) / total
        s_off = _interp_state(results, "S_off", time_h)
        e_off = _interp_state(results, "E_off", time_h)
        return 100.0 * e_off / (s_off + e_off + eps)

    if assay == "local_bystander_amplicon_pct":
        local = _interp_state(results, "B_local_bystander", time_h)
        apoe4 = _interp_state(results, "S_APOE4", time_h)
        apoe3 = _interp_state(results, "S_APOE3_like", time_h)
        apoe2 = _interp_state(results, "S_APOE2_like", time_h)
        mixed = _interp_state(results, "S_APOE158_only", time_h)
        return 100.0 * local / (apoe4 + apoe3 + apoe2 + mixed + local + eps)

    if assay == "puf_mismatch_offtarget_pct":
        s_off = _interp_state(results, "S_puf_off", time_h)
```





```
        e_off = _interp_state(results, "E_puf_off", time_h)
        return 100.0 * e_off / (s_off + e_off + eps)

    if assay == "deaminase_background_pct":
        s_bg = _interp_state(results, "S_deaminase_bg", time_h)
        e_bg = _interp_state(results, "E_deaminase_bg", time_h)
        return 100.0 * e_bg / (s_bg + e_bg + eps)

    if assay == "viability_pct":
        # A first-pass, interpretable burden proxy. Liver exposure and high
        # editor expression lower the score. Wet-lab viability data can replace
        # this proxy by fitting the coefficients later.
        liver = _interp_state(results, "A_liver", time_h)
        editor = _interp_state(results, "P_brain", time_h)
        burden = 0.15 * liver + 0.05 * editor
        return 100.0 * math.exp(-burden)

    raise KeyError(f"Unknown wet-lab assay '{assay}'.")


def load_wetlab_observations(path: str | Path) -> List[WetlabObservation]:
    """Load observations from the CSV template in wetlab/templates/."""
    observations: List[WetlabObservation] = []
    with open(path, "r", encoding="utf-8", newline="") as f:
        reader = csv.DictReader(f)
        for row in reader:
            observations.append(
                WetlabObservation(
                    assay=row["assay"],
                    time_h=float(row["time_h"]),
                    value=float(row["value"]),
                    sd=float(row.get("sd") or 1.0),
                    weight=float(row.get("weight") or 1.0),
                )
            )
    return observations


def residual_table(
    results: Dict, observations: Iterable[WetlabObservation]
) -> List[Dict[str, float | str]]:
    """
    Build a row-by-row residual table for calibration and wiki reporting.
    """
    rows: List[Dict[str, float | str]] = []
    for obs in observations:
        pred = simulated_assay_value(results, obs.assay, obs.time_h)
        sigma = max(float(obs.sd), 1e-12)
        residual = (pred - obs.value) / sigma
        rows.append(
            {
                "assay": obs.assay,
                "time_h": obs.time_h,
                "observed": obs.value,
                "predicted": pred,
                "sd": obs.sd,
                "weight": obs.weight,
                "weighted_residual": obs.weight * residual,
            }
        )
    return rows
```





```


def weighted_rmse(results: Dict, observations: Iterable[WetlabObservation]) -> float:
    """
    A calibration objective that can drive future parameter fitting.

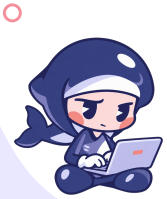
    Lower values mean the simulated delivery-expression-editing chain is closer
    to the wet-lab measurements after accounting for assay uncertainty.
    """
    rows = residual_table(results, observations)
    if not rows:
        return float("nan")
    squared = [float(row["weighted_residual"]) ** 2 for row in rows]
    return float(math.sqrt(sum(squared) / len(squared)))
```





models/__init__.py

```
1
```





utils/config_loader.py

```python
from __future__ import annotations

from pathlib import Path
from typing import Dict, Any
import yaml


def load_yaml(path: str | Path) -> Dict[str, Any]:
    with open(path, "r", encoding="utf-8") as f:
        data = yaml.safe_load(f)
    return data or {}


def load_base_config(project_root: str | Path) -> Dict[str, Any]:
    root = Path(project_root)
    return load_yaml(root / "config" / "base_config.yaml")
```





utils/io.py

```python
from __future__ import annotations

from pathlib import Path
from typing import Dict, Any
import json
import numpy as np


def ensure_dir(path: str | Path) -> Path:
    p = Path(path)
    p.mkdir(parents=True, exist_ok=True)
    return p


def save_results(results: Dict[str, Any], output_dir: str | Path) -> None:
    out = ensure_dir(output_dir)

    np.savez(
        out / "simulation_results.npz",
        t=results["t"],
        y=results["y"],
        state_order=np.array(results["state_order"], dtype=object),
    )

    metadata = {
        "dose": results["dose"],
        "route": results["config"].get("route", "footpad"),
        "n_states": len(results["state_order"]),
    }
    with open(out / "metadata.json", "w", encoding="utf-8") as f:
        json.dump(metadata, f, indent=2)
```





utils/plotting.py

```python
from __future__ import annotations

from pathlib import Path
from typing import Dict, Any
import matplotlib.pyplot as plt
import numpy as np

from models.pbpk.lymphatic_absorption import extract_absorption_flux_trajectories
from models.pbpk.organ_distribution import extract_distribution_flux_trajectories
from models.bbb.bbb_transport import extract_bbb_flux_trajectories
from models.intracellular.aav_intracellular import extract_intracellular_flux_trajectories
from models.editing.module5 import (
    extract_editing_flux_trajectories,
    compute_editing_metrics,
)


def _ensure_output_dir(output_dir: str | Path) -> Path:
    output_path = Path(output_dir)
    output_path.mkdir(parents=True, exist_ok=True)
    return output_path


def _has_state(idx: Dict[str, int], name: str) -> bool:
    return name in idx


def plot_module1_states(results: Dict[str, Any], output_dir: str | Path) -> Path:
    out = _ensure_output_dir(output_dir)
    t = results["t"]
    y = results["y"]
    idx = results["idx"]

    plt.figure(figsize=(8, 5))

    for state_name in ["A_dep", "A_lymph", "A_blood"]:
        if _has_state(idx, state_name):
            plt.plot(t, y[idx[state_name]], label=state_name)

    plt.xlabel("Time (h)")
    plt.ylabel("Amount")
    plt.title("Module 1: Absorption States")
    plt.legend()
    plt.tight_layout()

    path = out / "module1_states.png"
    plt.savefig(path, dpi=160)
    plt.close()
    return path


def plot_module1_fluxes(results: Dict[str, Any], output_dir: str | Path) -> Path:
    out = _ensure_output_dir(output_dir)
    t = results["t"]
    y = results["y"]
    idx = results["idx"]
```





```
    config = results["config"]
    route = config.get("route", "footpad")
    params = config["absorption"][route]

    # 这里用位置参数最稳，避免关键字参数名不一致
    fluxes = extract_absorption_flux_trajectories(t, y, params, idx)

    plt.figure(figsize=(8, 5))
    for name, arr in fluxes.items():
        plt.plot(t, arr, label=name)

    plt.xlabel("Time (h)")
    plt.ylabel("Flux")
    plt.title("Module 1: Absorption Fluxes")
    plt.legend()
    plt.tight_layout()

    path = out / "module1_fluxes.png"
    plt.savefig(path, dpi=160)
    plt.close()
    return path


def plot_module2_states(results: Dict[str, Any], output_dir: str | Path) -> Path:
    out = _ensure_output_dir(output_dir)
    t = results["t"]
    y = results["y"]
    idx = results["idx"]

    plt.figure(figsize=(9, 5))

    for state_name in [
        "A_blood",
        "A_liver",
        "A_peripheral",
        "A_brain_blood",
        "A_cleared",
    ]:
        if _has_state(idx, state_name):
            plt.plot(t, y[idx[state_name]], label=state_name)

    plt.xlabel("Time (h)")
    plt.ylabel("Amount")
    plt.title("Module 2: Systemic Distribution States")
    plt.legend()
    plt.tight_layout()

    path = out / "module2_states.png"
    plt.savefig(path, dpi=160)
    plt.close()
    return path


def plot_module2_fluxes(results: Dict[str, Any], output_dir: str | Path) -> Path:
    out = _ensure_output_dir(output_dir)
    t = results["t"]
    y = results["y"]
    idx = results["idx"]

    config = results["config"]
```





```
    params = config["distribution"]

    fluxes = extract_distribution_flux_trajectories(
        t,
        y,
        params,
        idx,
    )

    plt.figure(figsize=(10, 6))
    for name, arr in fluxes.items():
        plt.plot(t, arr, label=name)

    plt.xlabel("Time (h)")
    plt.ylabel("Flux")
    plt.title("Module 2: Distribution Fluxes")
    plt.legend()
    plt.tight_layout()

    path = out / "module2_fluxes.png"
    plt.savefig(path, dpi=160)
    plt.close()
    return path


def plot_module3_states(results: Dict[str, Any], output_dir: str | Path) -> Path:
    out = _ensure_output_dir(output_dir)
    t = results["t"]
    y = results["y"]
    idx = results["idx"]

    plt.figure(figsize=(9, 5))

    for state_name in ["A_brain_blood", "A_brain_EC", "A_brain_endo", "A_brain_ISF"]:
        if _has_state(idx, state_name):
            plt.plot(t, y[idx[state_name]], label=state_name)

    plt.xlabel("Time (h)")
    plt.ylabel("Amount")
    plt.title("Module 3: BBB Transport States")
    plt.legend()
    plt.tight_layout()

    path = out / "module3_states.png"
    plt.savefig(path, dpi=160)
    plt.close()
    return path


def plot_module3_fluxes(results: Dict[str, Any], output_dir: str | Path) -> Path:
    out = _ensure_output_dir(output_dir)
    t = results["t"]
    y = results["y"]
    idx = results["idx"]

    config = results["config"]
    params = config["bbb"]

    fluxes = extract_bbb_flux_trajectories(
        t,
        y,
```





```
        params,
        idx,
    )

    plt.figure(figsize=(10, 6))
    for name, arr in fluxes.items():
        plt.plot(t, arr, label=name)

    plt.xlabel("Time (h)")
    plt.ylabel("Flux")
    plt.title("Module 3: BBB Transport Fluxes")
    plt.legend()
    plt.tight_layout()

    path = out / "module3_fluxes.png"
    plt.savefig(path, dpi=160)
    plt.close()
    return path


def plot_module4_states(results: Dict[str, Any], output_dir: str | Path) -> Path:
    out = _ensure_output_dir(output_dir)
    t = results["t"]
    y = results["y"]
    idx = results["idx"]

    plt.figure(figsize=(9, 5))

    for state_name in [
        "A_brain_ISF",
        "A_brain_cell",
        "A_brain_nuc",
        "mRNA_brain",
        "P_brain",
    ]:
        if _has_state(idx, state_name):
            plt.plot(t, y[idx[state_name]], label=state_name)

    plt.xlabel("Time (h)")
    plt.ylabel("Amount / Level")
    plt.title("Module 4: Brain Uptake and Expression States")
    plt.legend()
    plt.tight_layout()

    path = out / "module4_states.png"
    plt.savefig(path, dpi=160)
    plt.close()
    return path


def plot_module4_fluxes(results: Dict[str, Any], output_dir: str | Path) -> Path:
    out = _ensure_output_dir(output_dir)
    t = results["t"]
    y = results["y"]
    idx = results["idx"]

    config = results["config"]
    params = config["intracellular"]

    fluxes = extract_intracellular_flux_trajectories(
        t,
```





```
        y,
        params,
        idx,
    )

    plt.figure(figsize=(10, 6))
    for name, arr in fluxes.items():
        plt.plot(t, arr, label=name)

    plt.xlabel("Time (h)")
    plt.ylabel("Flux")
    plt.title("Module 4: Brain Uptake and Expression Fluxes")
    plt.legend()
    plt.tight_layout()

    path = out / "module4_fluxes.png"
    plt.savefig(path, dpi=160)
    plt.close()
    return path


def plot_module5_states(results: Dict[str, Any], output_dir: str | Path) -> Path:
    out = _ensure_output_dir(output_dir)
    t = results["t"]
    y = results["y"]
    idx = results["idx"]

    plt.figure(figsize=(9, 5))

    state_names = [
        "P_brain",
        "S_APOE4",
        "S_APOE3_like",
        "S_APOE2_like",
        "S_APOE158_only",
        "B_local_bystander",
        "S_puf_off",
        "E_puf_off",
        "S_deaminase_bg",
        "E_deaminase_bg",
        "S_on",
        "S_off",
        "ES_on",
        "ES_off",
        "E_on",
        "E_off",
    ]
    for state_name in state_names:
        if _has_state(idx, state_name):
            plt.plot(t, y[idx[state_name]], label=state_name)

    plt.xlabel("Time (h)")
    plt.ylabel("Amount / Level")
    plt.title("Module 5: PUF-APOBEC Editing States")
    plt.legend()
    plt.tight_layout()

    path = out / "module5_states.png"
    plt.savefig(path, dpi=160)
    plt.close()
    return path
```





```python


def plot_module5_fluxes(results: Dict[str, Any], output_dir: str | Path) -> Path:
    out = _ensure_output_dir(output_dir)
    t = results["t"]
    y = results["y"]
    idx = results["idx"]

    config = results["config"]
    params = config["editing"]

    fluxes = extract_editing_flux_trajectories(
        t,
        y,
        params,
        idx,
    )

    plt.figure(figsize=(9, 5))
    for name, arr in fluxes.items():
        plt.plot(t, arr, label=name)

    plt.xlabel("Time (h)")
    plt.ylabel("Flux")
    plt.title("Module 5: PUF–APOBEC Editing Fluxes")
    plt.legend()
    plt.tight_layout()

    path = out / "module5_fluxes.png"
    plt.savefig(path, dpi=160)
    plt.close()
    return path


def plot_module5_metrics(results: Dict[str, Any], output_dir: str | Path) -> Path:
    out = _ensure_output_dir(output_dir)
    t = results["t"]
    y = results["y"]
    idx = results["idx"]
    config = results["config"]

    metrics = compute_editing_metrics(y=y, idx=idx, config=config)

    plt.figure(figsize=(9, 5))
    plt.plot(t, metrics["on_target_editing_rate"], label="on_target_editing_rate")
    plt.plot(t, metrics["off_target_burden"], label="off_target_burden")
    plt.plot(t, metrics["specificity_index"], label="specificity_index")
    if "on_target_editing_fraction" in metrics:
        plt.plot(
            t, metrics["on_target_editing_fraction"], label="on_target_editing_fraction"
        )
    if "off_target_editing_fraction" in metrics:
        plt.plot(
            t, metrics["off_target_editing_fraction"], label="off_target_editing_fraction"
        )
    for name in [
        "apoe4_fraction",
        "apoe3_like_fraction",
        "apoe2_like_fraction",
        "mixed_edit_fraction",
        "local_bystander_burden",
```





```
        "puf_mismatch_burden",
        "deaminase_background_burden",
        "ldlr_binding_risk_proxy",
    ]:
        if name in metrics:
            plt.plot(t, metrics[name], label=name)

    plt.xlabel("Time (h)")
    plt.ylabel("Metric")
    plt.title("Module 5: Editing Metrics")
    plt.legend()
    plt.tight_layout()

    path = out / "module5_metrics.png"
    plt.savefig(path, dpi=160)
    plt.close()
    return path


def plot_combined_states(results: Dict[str, Any], output_dir: str | Path) -> Path:
    """
    一张总览图，把模块一和模块二关键状态放一起。
    """
    out = _ensure_output_dir(output_dir)
    t = results["t"]
    y = results["y"]
    idx = results["idx"]

    selected_states = [
        "A_dep",
        "A_lymph",
        "A_blood",
        "A_liver",
        "A_peripheral",
        "A_brain_blood",
        "A_brain_EC",
        "A_brain_endo",
        "A_brain_ISF",
        "A_brain_cell",
        "A_brain_nuc",
        "mRNA_brain",
        "P_brain",
        "S_APOE4",
        "S_APOE3_like",
        "S_APOE2_like",
        "S_APOE158_only",
        "B_local_bystander",
        "E_puf_off",
        "E_deaminase_bg",
        "S_on",
        "S_off",
        "ES_on",
        "ES_off",
        "E_on",
        "E_off",
        "A_cleared",
    ]

    plt.figure(figsize=(10, 6))
    for state_name in selected_states:
        if _has_state(idx, state_name):
```



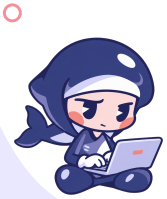


```
            plt.plot(t, y[idx[state_name]], label=state_name)

    plt.xlabel("Time (h)")
    plt.ylabel("Amount")
    plt.title("Key State Trajectories")
    plt.legend()
    plt.tight_layout()

    path = out / "combined_states.png"
    plt.savefig(path, dpi=160)
    plt.close()
    return path


def plot_all(results: Dict[str, Any], output_dir: str | Path) -> Dict[str, Path]:
    figure_paths = {
        "module1_states": plot_module1_states(results, output_dir),
        "module1_fluxes": plot_module1_fluxes(results, output_dir),
        "module2_states": plot_module2_states(results, output_dir),
        "module2_fluxes": plot_module2_fluxes(results, output_dir),
        "module3_states": plot_module3_states(results, output_dir),
        "module3_fluxes": plot_module3_fluxes(results, output_dir),
        "module4_states": plot_module4_states(results, output_dir),
        "module4_fluxes": plot_module4_fluxes(results, output_dir),
        "module5_states": plot_module5_states(results, output_dir),
        "module5_fluxes": plot_module5_fluxes(results, output_dir),
        "module5_metrics": plot_module5_metrics(results, output_dir),
        "combined_states": plot_combined_states(results, output_dir),
    }
    return figure_paths
```





config/base_config.yaml

```yaml
route: footpad

simulation:
  default_t_end: 48.0
  default_dt: 0.1
  solver: LSODA

absorption:
  footpad:
    k_lymph: 0.08
    k_blood: 0.03
    k_deg_loc: 0.005
    k_lymph_to_blood: 0.12
  im:
    # Literature-informed route prior; not calibrated to a specific capsid.
    k_lymph: 0.04
    k_blood: 0.05
    k_deg_loc: 0.006
    k_lymph_to_blood: 0.10
  iv:
    # IV dose is initialized directly in blood; no local depot flux remains.
    k_lymph: 0.0
    k_blood: 0.0
    k_deg_loc: 0.0
    k_lymph_to_blood: 0.0

distribution:
  k_blood_to_liver: 0.08
  k_blood_to_peripheral: 0.05
  k_blood_to_brain: 0.01

  k_liver_to_blood: 0.01
  k_peripheral_to_blood: 0.02
  k_brain_to_blood: 0.0

  k_clear_blood: 0.01
  k_clear_liver: 0.02
  k_clear_peripheral: 0.01
  k_clear_brainblood: 0.001

bbb:
  k_brainblood_to_EC: 0.02
  k_EC_to_endo: 0.05
  k_EC_to_brainblood: 0.01

  k_endo_to_ISF: 0.01
  k_endo_to_brainblood: 0.005
  k_endo_loss: 0.01

  k_clear_ISF: 0.001

intracellular:
  k_ISF_to_cell: 0.03
  k_cell_to_nuc: 0.02
  k_cell_loss: 0.005

```





```yaml
  k_deg_v: 0.003
  k_tx: 0.4
  k_deg_m: 0.08
  k_tl: 0.9
  k_deg_p: 0.03

editing:
  # Module 5 can be switched between:
  #   competitive              : original simplified competition model
  #   mechanistic_puf_apobec   : explicit E + S <-> ES -> E + P model
  #   apoe_multisite           : v4 APOE112/APOE158-aware model
  model: apoe_multisite

  # v4 explicitly records the deaminase identity. This prevents mixing
  # APOBEC3A, APOBEC1, engineered A3A, and ProAPOBEC assumptions.
  editor_type: A3A

  # Original competitive-editing parameters are kept so the old model can
  # still run by setting model: competitive.
  k_cat_on: 0.5
  K_d_on: 0.2
  k_cat_off: 0.05
  K_d_off: 1.0

  # Mechanistic PUF-APOBEC parameters.
  # P_brain from module 4 is treated as free active editor E.
  # On-target branch: E + APOE4 RNA <-> ES_on -> E + edited APOE2-like RNA.
  k_on_on: 1.0
  k_off_on: 0.5
  # Off-target branch uses weaker/slower binding and catalysis by default.
  k_on_off: 0.2
  k_off_off: 1.0

  # RNA turnover terms. These keep substrate pools biologically replenished;
  # with no editor, S_on and S_off tend to k_prod / k_deg.
  k_prod_on: 0.1
  k_deg_on: 0.1
  k_prod_off: 0.5
  k_deg_off: 0.1

  k_loss_on: 0.0
  k_loss_off: 0.0

  S_on_init: 1.0
  S_off_init: 5.0
  specificity_eps: 1.0e-12

  # v4 APOE biology: APOE4 is Arg112/Arg158; editing 112 only gives
  # APOE3-like, while editing both 112 and 158 gives APOE2-like.
  S_APOE4_init: 1.0
  S_puf_off_init: 1.5
  S_deaminase_bg_init: 4.0

  k_on_112: 1.0
  k_off_112: 0.5
  # gkac713 reports CU-REWIRE C-to-U editing in HEK293T at 48 h, with strong
  # activity when the edited C is in the preferred window/context. These are
  # normalized effective priors, not literal per-second biochemical constants.
  k_cat_112: 1.0
  k_on_158: 0.35
  k_off_158: 0.7
```





```
    k_cat_158: 0.15

    k_prod_apoe: 0.1
    k_deg_apoe: 0.1

    local_bystander_per_112: 0.04
    local_bystander_per_158: 0.06

    # 10-repeat PUF designs are used as the default v4 prior because gkac713
    # reports substantially improved transcriptome specificity versus shorter
    # recognition designs.
    k_on_puf_off: 0.015
    k_off_puf_off: 1.0
    k_cat_puf_off: 0.006

    # Background editing remains nonzero for active A3A-family constructs but is
    # separated from PUF-mediated off-target editing and set to zero for inactive
    # controls through editor_activity_scale in the design table.
    k_deaminase_bg: 0.0003
    k_prod_puf_off: 0.15
    k_deg_puf_off: 0.1
    k_prod_deaminase_bg: 0.4
    k_deg_deaminase_bg: 0.1

    uc_context_scale: 1.0
    a3a_background_scale: 0.8
    apobec1_cat_scale: 0.7
    apobec1_background_scale: 0.4
    proapobec_cat_scale: 1.25
    proapobec_background_scale: 0.5

optimization:
    E_on_target_threshold: 0.2
    E_off_max: 0.05
    SI_min: 5.0
    AUC_liver_max: 5.0
    Cmax_blood_max: 1.0

    dose_scan:
      dose_min: 0.2
      dose_max: 2.4
      n_doses: 23
```





config/parameter_provenance.yaml

```yaml
# Parameter provenance for the iterated iGEM delivery-expression-editing model.
#
# Units are the model's normalized amount/hour convention unless stated
# otherwise. Because the wet-lab system is not calibrated yet, ranges are used
# as uncertainty priors rather than claims of exact biological constants.

parameters:
  absorption.footpad.k_lymph:
    nominal: 0.08
    low: 0.02
    high: 0.20
    distribution: log_uniform
    confidence: medium
    rationale: Local depot drainage is route- and formulation-dependent; keep broad until dose-time qPCR data are available.
    source: Original combined model, to be calibrated with local injection-site and blood qPCR time course.

  absorption.footpad.k_blood:
    nominal: 0.03
    low: 0.005
    high: 0.10
    distribution: log_uniform
    confidence: low
    rationale: Direct vascular entry from local administration is uncertain; broad prior prevents overconfidence.
    source: Original combined model plus route-dependence assumption.

  distribution.k_blood_to_liver:
    nominal: 0.08
    low: 0.02
    high: 0.30
    distribution: log_uniform
    confidence: medium
    rationale: Liver is a major systemic sink for AAV after systemic exposure; range should be narrowed with liver vector-genome qPCR.
    source: AAV gene-therapy vector biodistribution literature and AAV tropism summaries.

  distribution.k_blood_to_brain:
    nominal: 0.01
    low: 0.001
    high: 0.05
    distribution: log_uniform
    confidence: medium
    rationale: AAV9-like brain entry is much lower than liver uptake, but nonzero because AAV9 can cross the BBB.
    source: AAV9 BBB-crossing studies and AAV serotype tropism summaries.

  bbb.k_brainblood_to_EC:
    nominal: 0.02
    low: 0.002
    high: 0.08
    distribution: log_uniform
    confidence: low
    rationale: Captures endothelial association; should be calibrated from brain vascular versus parenchymal vector measurements.
```





```
    source: Mechanistic BBB transcytosis abstraction.

  bbb.k_endo_to_ISF:
    nominal: 0.01
    low: 0.001
    high: 0.04
    distribution: log_uniform
    confidence: low
    rationale: Successful transcytosis is a key uncertainty; use a broad prior and target with bra
in-delivery assays.
    source: BBB transcytosis and AAV9 CNS delivery literature.

  intracellular.k_tx:
    nominal: 0.4
    low: 0.05
    high: 1.20
    distribution: log_uniform
    confidence: medium
    rationale: Normalized transcription rate after vector reaches nucleus; calibrate with PUF-APOB
EC qPCR.
    source: Original model, bounded by expected mammalian expression time-course behavior.

  intracellular.k_deg_m:
    nominal: 0.08
    low: 0.02
    high: 0.25
    distribution: log_uniform
    confidence: medium
    rationale: Corresponds to an mRNA half-life of roughly 2.8-35 h; mammalian mRNA half-lives spa
n minutes to days.
    source: Mammalian mRNA turnover literature.

  intracellular.k_tl:
    nominal: 0.9
    low: 0.10
    high: 2.50
    distribution: log_uniform
    confidence: medium
    rationale: Normalized translation rate; to be calibrated with Western blot or tagged-editor fl
uorescence.
    source: Original model, wet-lab calibration needed.

  intracellular.k_deg_p:
    nominal: 0.03
    low: 0.005
    high: 0.10
    distribution: log_uniform
    confidence: medium
    rationale: Protein half-life prior spans about 7-139 h, covering short-lived fused editors and
 more stable proteins.
    source: General mammalian protein turnover literature.

  editing.k_on_on:
    nominal: 1.0
    low: 0.10
    high: 10.0
    distribution: log_uniform
    confidence: low
    rationale: Effective binding rate for engineered PUF-target RNA interaction in normalized unit
s; sequence-specific and must be fitted.
    source: PUF-RNA binding literature; current value is normalized for model-scale exploration.
```





```
  editing.k_off_on:
    nominal: 0.5
    low: 0.05
    high: 2.0
    distribution: log_uniform
    confidence: low
    rationale: Effective dissociation rate; should be constrained by EMSA/SPR or editing time-course data.
    source: PUF-RNA binding literature; current value is normalized for model-scale exploration.

  editing.k_cat_on:
    nominal: 0.5
    low: 0.05
    high: 2.0
    distribution: log_uniform
    confidence: low
    rationale: Effective C-to-U editing turnover; captures APOBEC chemistry plus RNA accessibility.
    source: APOBEC/RNA-editing literature; fitted wet-lab amplicon editing time course required.

  editing.k_on_off:
    nominal: 0.2
    low: 0.01
    high: 2.0
    distribution: log_uniform
    confidence: low
    rationale: Off-target binding prior is lower than on-target but uncertain; sequence screening should refine this.
    source: Off-target risk model assumption informed by RNA-binding specificity.

  editing.k_cat_off:
    nominal: 0.05
    low: 0.001
    high: 0.5
    distribution: log_uniform
    confidence: low
    rationale: Off-target catalytic efficiency is expected to be lower but not zero; amplicon/RNA-seq validation should update.
    source: Off-target editing risk assumption.

  editing.k_on_112:
    nominal: 1.0
    low: 0.10
    high: 10.0
    distribution: log_uniform
    confidence: low
    rationale: Effective PUF binding rate to APOE codon-112 target window, sequence/accessibility-adjusted in v4.
    source: Han et al. NAR 2022 gkac713 CU-REWIRE design rules; to be calibrated with APOE112 amplicon time course.

  editing.k_cat_112:
    nominal: 1.0
    low: 0.03
    high: 3.0
    distribution: log_uniform
    confidence: low
    rationale: Effective APOE112 C-to-U editing turnover; gkac713 reports strong CU-REWIRE activity in the preferred window/context at 48 h, so v4 uses a higher APOE112 prior before wet-lab fitting.
```





```
    source: Han et al. NAR 2022 gkac713 CU-REWIRE HEK293T editing results.

  editing.k_on_158:
    nominal: 0.35
    low: 0.02
    high: 5.0
    distribution: log_uniform
    confidence: low
    rationale: Effective PUF binding/capture rate for the APOE158 edit window, expected to be lowe
r unless explicitly targeted.
    source: v4 dual-site editing prior.

  editing.k_cat_158:
    nominal: 0.15
    low: 0.005
    high: 2.0
    distribution: log_uniform
    confidence: low
    rationale: Effective APOE158 turnover is lower by default because the main therapeutic design
objective is APOE4-to-APOE3-like editing at codon 112; dual editing remains tracked as APOE2-like
risk.
    source: Han et al. NAR 2022 gkac713 editing-window logic plus APOE therapeutic-objective choic
e.

  editing.k_on_puf_off:
    nominal: 0.015
    low: 0.001
    high: 0.5
    distribution: log_uniform
    confidence: low
    rationale: Aggregate PUF-mismatch off-target binding rate; lowered for the default 10-repeat P
UF prior because gkac713 reports substantially improved transcriptome specificity with expanded PU
F recognition.
    source: Han et al. NAR 2022 gkac713 10R REWIRE specificity results.

  editing.k_cat_puf_off:
    nominal: 0.006
    low: 0.0001
    high: 0.2
    distribution: log_uniform
    confidence: low
    rationale: Aggregate PUF-mediated off-target catalytic rate after mismatch binding; default is
 reduced for 10R recognition and then rescaled by each design row.
    source: Han et al. NAR 2022 gkac713 off-target comparison.

  editing.k_deaminase_bg:
    nominal: 0.0003
    low: 0.00001
    high: 0.02
    distribution: log_uniform
    confidence: low
    rationale: Deaminase-only background editing rate independent of correct PUF targeting; active
 A3A-family constructs keep a nonzero prior, while inactive/PUF-only controls set this pathway to
zero.
    source: Han et al. NAR 2022 gkac713 free-A3A/off-target controls; should be constrained with i
nactive/PUF-only controls.

  editing.local_bystander_per_112:
    nominal: 0.04
    low: 0.001
    high: 0.2
```





```
    distribution: log_uniform
    confidence: low
    rationale: Nearby C-edit burden caused after correct APOE112 targeting; CU-REWIRE has a narrow
er window than broader deaminase systems but local bystander edits remain possible.
    source: Han et al. NAR 2022 gkac713 local/windowed editing behavior.

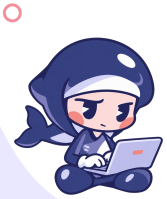
  editing.local_bystander_per_158:
    nominal: 0.06
    low: 0.001
    high: 0.4
    distribution: log_uniform
    confidence: low
    rationale: Nearby C-edit burden around APOE158, tracked separately because the 158 edit change
s LDLR-binding interpretation.
    source: v4 local bystander prior.
```





optimization/iterated/__init__.py

```
"""Iterated-model analysis utilities."""
```





optimization/iterated/uncertainty.py

```python
from __future__ import annotations

from copy import deepcopy
from pathlib import Path
from typing import Dict, Iterable, Tuple
import csv
import math
import random

import yaml

from models.full_model.simulator import run_simulation
from optimization.metrics import extract_summary_metrics


def _get_nested(config: Dict, path: str):
    node = config
    parts = path.split(".")
    for part in parts:
        node = node[part]
    return node


def _set_nested(config: Dict, path: str, value: float) -> None:
    node = config
    parts = path.split(".")
    for part in parts[:-1]:
        node = node[part]
    node[parts[-1]] = float(value)


def load_parameter_priors(path: str | Path) -> Dict:
    with open(path, "r", encoding="utf-8") as f:
        data = yaml.safe_load(f) or {}
    return data.get("parameters", {})


def sample_parameter(prior: Dict, rng: random.Random) -> float:
    low = float(prior["low"])
    high = float(prior["high"])
    distribution = prior.get("distribution", "uniform")
    if distribution == "log_uniform":
        return math.exp(rng.uniform(math.log(low), math.log(high)))
    if distribution == "uniform":
        return rng.uniform(low, high)
    raise ValueError(f"Unsupported prior distribution: {distribution}")


def sample_config(
    base_config: Dict, priors: Dict, rng: random.Random
) -> Tuple[Dict, Dict[str, float]]:
    """
    Draw one uncertain parameter set and apply it to a copy of the config.
    """
    config = deepcopy(base_config)
    sampled = {}
```





```
    for path, prior in priors.items():
        if prior.get("enabled", True) is False:
            continue
        # Skip missing paths gracefully so the provenance table can include
        # future parameters before the model code uses them.
        try:
            _get_nested(config, path)
        except KeyError:
            continue
        value = sample_parameter(prior, rng)
        _set_nested(config, path, value)
        sampled[path] = value
    return config, sampled


def run_uncertainty_scan(
    base_config: Dict,
    priors: Dict,
    n_samples: int,
    dose: float,
    t_end: float,
    dt: float,
    seed: int = 7,
) -> Iterable[Dict[str, float]]:
    """
    Monte Carlo scan over uncertain biological parameters.

    The output rows include both sampled parameters and decision metrics. This
    makes the analysis useful for iGEM-style "what should we measure next?"
    reasoning, because high-spread metrics point to weakly constrained biology.
    """
    rng = random.Random(seed)
    for sample_id in range(int(n_samples)):
        config, sampled = sample_config(base_config, priors, rng)
        sim = run_simulation(config=config, dose=dose, t_end=t_end, dt=dt)
        summary = extract_summary_metrics(sim)
        yield {
            "sample_id": sample_id,
            "dose": dose,
            **sampled,
            **summary,
        }


def save_rows(rows: Iterable[Dict], output_path: str | Path) -> Path:
    rows = list(rows)
    output_path = Path(output_path)
    output_path.parent.mkdir(parents=True, exist_ok=True)
    fieldnames = sorted({key for row in rows for key in row.keys()})
    with open(output_path, "w", encoding="utf-8", newline="") as f:
        writer = csv.DictWriter(f, fieldnames=fieldnames)
        writer.writeheader()
        writer.writerows(rows)
    return output_path
```





optimization/metrics.py

```python
from __future__ import annotations

from typing import Dict, Any
import numpy as np

from models.editing.module5 import compute_editing_metrics


def trapz_auc(t: np.ndarray, y: np.ndarray) -> float:
    return float(np.trapezoid(y, t))


def extract_summary_metrics(results: Dict[str, Any]) -> Dict[str, float]:
    """
    Extract summary metrics from a full simulation result.

    Returns
    -------
    dict
        Summary metrics for module 6.
    """
    t = results["t"]
    y = results["y"]
    idx = results["idx"]
    config = results["config"]

    metrics = compute_editing_metrics(y=y, idx=idx, config=config)

    E_on_final = float(y[idx["E_on"], -1])
    E_off_final = float(y[idx["E_off"], -1])

    on_target_editing_rate_final = float(metrics["on_target_editing_rate"][-1])
    off_target_burden_final = float(metrics["off_target_burden"][-1])
    specificity_index_final = float(metrics["specificity_index"][-1])

    P_brain_peak = float(np.max(y[idx["P_brain"], :]))
    AUC_liver = float(trapz_auc(t, y[idx["A_liver"], :]))
    Cmax_blood = float(np.max(y[idx["A_blood"], :]))

    summary = {
        "E_on_final": E_on_final,
        "E_off_final": E_off_final,
        "on_target_editing_rate_final": on_target_editing_rate_final,
        "off_target_burden_final": off_target_burden_final,
        "specificity_index_final": specificity_index_final,
        "P_brain_peak": P_brain_peak,
        "AUC_liver": AUC_liver,
        "Cmax_blood": Cmax_blood,
    }
    for name, arr in metrics.items():
        if name not in {
            "on_target_editing_rate",
            "off_target_burden",
            "specificity_index",
        }:
            summary[f"{name}_final"] = float(arr[-1])
    return summary
```





optimization/min_effective_dose.py

```python
from __future__ import annotations

from typing import Dict, Any, List
import copy
import numpy as np

from models.full_model.simulator import run_simulation
from optimization.metrics import extract_summary_metrics
from optimization.therapeutic_window import evaluate_therapeutic_window


def build_dose_grid(config: Dict) -> np.ndarray:
    scan_cfg = config["optimization"]["dose_scan"]
    return np.linspace(
        float(scan_cfg["dose_min"]),
        float(scan_cfg["dose_max"]),
        int(scan_cfg["n_doses"]),
    )


def run_dose_scan(
    base_config: Dict,
    t_end: float | None = None,
    dt: float | None = None,
) -> List[Dict[str, Any]]:
    """
    Run dose scan across a configured dose grid.
    """
    dose_grid = build_dose_grid(base_config)

    if t_end is None:
        t_end = float(base_config["simulation"]["default_t_end"])
    if dt is None:
        dt = float(base_config["simulation"]["default_dt"])

    scan_results = []

    for dose in dose_grid:
        config = copy.deepcopy(base_config)

        sim = run_simulation(
            config=config,
            dose=float(dose),
            t_end=t_end,
            dt=dt,
        )

        summary = extract_summary_metrics(sim)
        feasibility = evaluate_therapeutic_window(summary, config)

        row = {
            "dose": float(dose),
            **summary,
            **feasibility,
        }
        scan_results.append(row)
```





```
    return scan_results


def find_min_effective_dose(scan_results: List[Dict[str, Any]]) -> Dict[str, Any] | None:
    """
    Find the minimum feasible dose from dose scan results.
    """
    feasible_rows = [row for row in scan_results if row["feasible"]]
    if not feasible_rows:
        return None
    feasible_rows = sorted(feasible_rows, key=lambda x: x["dose"])
    return feasible_rows[0]
```





optimization/report_module6.py

```python
from __future__ import annotations

from pathlib import Path
from typing import List, Dict, Any
import pandas as pd
import matplotlib.pyplot as plt


def _ensure_dir(path: str | Path) -> Path:
    path = Path(path)
    path.mkdir(parents=True, exist_ok=True)
    return path


def save_dose_scan_table(
    scan_results: List[Dict[str, Any]], output_dir: str | Path
) -> Path:
    out = _ensure_dir(output_dir)
    df = pd.DataFrame(scan_results)
    path = out / "module6_dose_scan.csv"
    df.to_csv(path, index=False)
    return path


def plot_dose_response(
    scan_results: List[Dict[str, Any]], output_dir: str | Path
) -> Dict[str, Path]:
    out = _ensure_dir(output_dir)
    df = pd.DataFrame(scan_results)

    figure_paths = {}

    plot_specs = [
        (
            "on_target_editing_rate_final",
            "Dose vs On-target Editing Rate",
            "dose_vs_on_target.png",
        ),
        (
            "off_target_burden_final",
            "Dose vs Off-target Burden",
            "dose_vs_off_target.png",
        ),
        (
            "specificity_index_final",
            "Dose vs Specificity Index",
            "dose_vs_specificity.png",
        ),
        ("AUC_liver", "Dose vs Liver AUC", "dose_vs_auc_liver.png"),
        ("Cmax_blood", "Dose vs Blood Cmax", "dose_vs_cmax_blood.png"),
        ("P_brain_peak", "Dose vs Brain Expression Peak", "dose_vs_pbrain_peak.png"),
    ]

    for metric_name, title, filename in plot_specs:
        plt.figure(figsize=(7, 4.5))
        plt.plot(df["dose"], df[metric_name], marker="o")
```





```
        plt.xlabel("Dose")
        plt.ylabel(metric_name)
        plt.title(title)
        plt.tight_layout()
        path = out / filename
        plt.savefig(path, dpi=160)
        plt.close()
        figure_paths[metric_name] = path

    # Feasibility map
    plt.figure(figsize=(7, 4.5))
    colors = ["green" if x else "red" for x in df["feasible"]]
    plt.scatter(df["dose"], df["on_target_editing_rate_final"], c=colors)
    plt.xlabel("Dose")
    plt.ylabel("on_target_editing_rate_final")
    plt.title("Feasibility Map (green = feasible)")
    plt.tight_layout()
    path = out / "dose_feasibility_map.png"
    plt.savefig(path, dpi=160)
    plt.close()
    figure_paths["feasibility_map"] = path

    return figure_paths
```





optimization/therapeutic_window.py

```python
from __future__ import annotations

from typing import Dict


def evaluate_therapeutic_window(
    summary_metrics: Dict[str, float],
    config: Dict,
) -> Dict[str, object]:
    """
    Determine whether a simulation result is feasible under current thresholds.
    """
    opt_cfg = config["optimization"]

    on_target_ok = (
        summary_metrics["on_target_editing_rate_final"]
        >= opt_cfg["E_on_target_threshold"]
    )

    off_target_ok = summary_metrics["off_target_burden_final"] <= opt_cfg["E_off_max"]

    specificity_ok = summary_metrics["specificity_index_final"] >= opt_cfg["SI_min"]

    liver_ok = summary_metrics["AUC_liver"] <= opt_cfg["AUC_liver_max"]

    blood_ok = summary_metrics["Cmax_blood"] <= opt_cfg["Cmax_blood_max"]

    feasible = all(
        [
            on_target_ok,
            off_target_ok,
            specificity_ok,
            liver_ok,
            blood_ok,
        ]
    )

    return {
        "feasible": feasible,
        "on_target_ok": on_target_ok,
        "off_target_ok": off_target_ok,
        "specificity_ok": specificity_ok,
        "liver_ok": liver_ok,
        "blood_ok": blood_ok,
    }
```





scripts/run_design_uncertainty.py

```
"""
M7: Monte Carlo uncertainty intervals for the recommended design.

Applies the recommended PUF-APOBEC design (default: 10R-proapobec) on top of
SAMPLED parameter sets — i.e. sequence-derived kinetic modifiers are treated
as deterministic design features, while the underlying biological rate
constants remain uncertain (priors from config/parameter_provenance.yaml).

Reports median + 90% prediction intervals for the decision metrics and the
probability of meeting the pre-registered decision criteria.

Usage:
    python scripts/run_design_uncertainty.py \
        --design_id 10R-proapobec --dose 0.9 --t_end 168 --samples 200 \
        --out_dir <abs path>/new_results/design_uncertainty
"""

from __future__ import annotations

import argparse
import json
import math
import random
import sys
from pathlib import Path

ROOT = Path(__file__).resolve().parents[1]
sys.path.insert(0, str(ROOT))

from utils.config_loader import load_base_config  # noqa: E402
from models.full_model.simulator import run_simulation  # noqa: E402
from models.editing.sequence_to_kinetics import (  # noqa: E402
    load_design_table,
    apply_design_to_config,
)
from optimization.metrics import extract_summary_metrics  # noqa: E402
from optimization.iterated.uncertainty import (  # noqa: E402
    load_parameter_priors,
    sample_config,
)

METRICS = [
    "apoe3_like_fraction_final",
    "ldlr_binding_risk_proxy_final",
    "off_target_burden_final",
    "specificity_index_final",
    "on_target_editing_rate_final",
]

# Pre-registered decision criteria (direction: >= or <=).
CRITERIA = {
    "apoe3_like_fraction_final": (">=", 0.30, "desired product >= 30% of APOE RNA"),
    "specificity_index_final": (">=", 5.0, "specificity index >= 5"),
    "ldlr_binding_risk_proxy_final": ("<=", 0.01, "APOE2-like risk proxy <= 1%"),
}

```





```

def percentile(xs, q):
    xs = sorted(xs)
    if not xs:
        return float("nan")
    k = (len(xs) - 1) * q
    f = math.floor(k)
    c = math.ceil(k)
    if f == c:
        return xs[int(k)]
    return xs[f] + (xs[c] - xs[f]) * (k - f)


def main() -> None:
    parser = argparse.ArgumentParser(description="Design-specific MC uncertainty scan")
    parser.add_argument("--design_id", type=str, default="10R-proapobec")
    parser.add_argument("--dose", type=float, default=0.9)
    parser.add_argument("--t_end", type=float, default=168.0)
    parser.add_argument("--dt", type=float, default=0.2)
    parser.add_argument("--samples", type=int, default=200)
    parser.add_argument("--seed", type=int, default=7)
    parser.add_argument("--out_dir", type=str, required=True)
    args = parser.parse_args()

    out_dir = Path(args.out_dir)
    out_dir.mkdir(parents=True, exist_ok=True)

    base_config = load_base_config(ROOT)
    base_config["route"] = "footpad"
    priors = load_parameter_priors(ROOT / "config" / "parameter_provenance.yaml")
    designs = {
        r["design_id"]: r
        for r in load_design_table(
            ROOT / "wetlab" / "templates" / "puf_design_candidates_template.csv"
        )
    }
    design = designs[args.design_id]

    rng = random.Random(args.seed)
    rows = []
    for sample_id in range(int(args.samples)):
        sampled_config, sampled = sample_config(base_config, priors, rng)
        design_config, _ = apply_design_to_config(sampled_config, design)
        sim = run_simulation(
            config=design_config, dose=args.dose, t_end=args.t_end, dt=args.dt
        )
        summary = extract_summary_metrics(sim)
        rows.append({"sample_id": sample_id, **sampled, **summary})
        if (sample_id + 1) % 25 == 0:
            print(f"  {sample_id + 1}/{args.samples} samples done")

    # ---- summarize
    summary_out = {
        "design_id": args.design_id,
        "dose": args.dose,
        "t_end_h": args.t_end,
        "n_samples": args.samples,
        "seed": args.seed,
        "metrics": {},
    }
    for m in METRICS:
```





```
        xs = [
            float(r[m])
            for r in rows
            if r.get(m) is not None and math.isfinite(float(r[m]))
        ]
        if not xs:
            continue
        entry = {
            "median": percentile(xs, 0.5),
            "p05": percentile(xs, 0.05),
            "p95": percentile(xs, 0.95),
            "mean": sum(xs) / len(xs),
            "min": min(xs),
            "max": max(xs),
        }
        if m in CRITERIA:
            op, thr, label = CRITERIA[m]
            ok = [x >= thr if op == ">=" else x <= thr for x in xs]
            entry["criterion"] = label
            entry["criterion_threshold"] = thr
            entry["prob_meet_criterion"] = sum(ok) / len(ok)
        summary_out["metrics"][m] = entry

    # ---- write outputs
    import csv

    csv_path = out_dir / "design_uncertainty_samples.csv"
    fieldnames = sorted({k for r in rows for k in r.keys()})
    with open(csv_path, "w", newline="", encoding="utf-8") as f:
        writer = csv.DictWriter(f, fieldnames=fieldnames)
        writer.writeheader()
        writer.writerows(rows)

    json_path = out_dir / "design_uncertainty_summary.json"
    with open(json_path, "w", encoding="utf-8") as f:
        json.dump(summary_out, f, indent=2)

    md_path = out_dir / "design_uncertainty_summary.md"
    with open(md_path, "w", encoding="utf-8") as f:
        f.write(f"# Uncertainty intervals — recommended design `{args.design_id}`\n\n")
        f.write(
            f"Monte Carlo over uncertain biological parameters (n={args.samples}, "
            f"dose={args.dose}, t_end={args.t_end} h, footpad route). "
            "Sequence-derived design modifiers are applied on top of each sampled parameter set.\n
\n"
        )
        f.write("| metric | median | 90% PI (p05–p95) | criterion | P(meet) |\n")
        f.write("|--------|--------|------------------|-----------|---------|\n")
        for m in METRICS:
            e = summary_out["metrics"].get(m)
            if not e:
                continue
            crit = e.get("criterion", "—")
            p = f"{e['prob_meet_criterion']:.2f}" if "prob_meet_criterion" in e else "—"
            f.write(
                f"| {m} | {e['median']:.4g} | {e['p05']:.4g} – {e['p95']:.4g} | {crit} | {p} |\n"
            )

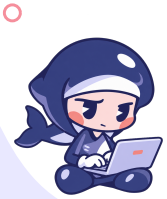
    print(f"\nDesign {args.design_id} @ dose {args.dose}, n={args.samples}")
    for m in METRICS:
        e = summary_out["metrics"].get(m)
```





```
        if e:
            p = (
                f", P(crit)={e['prob_meet_criterion']:.2f}"
                if "prob_meet_criterion" in e
                else ""
            )
            print(
                f"  {m:32s} median={e['median']:.4g}  90%PI=[{e['p05']:.4g}, {e['p95']:.4g}]{p}"
            )
    print(f"CSV: {csv_path}\nJSON: {json_path}\nMD: {md_path}")


if __name__ == "__main__":
    main()
```





scripts/run_iterated_analysis.py

```python
from __future__ import annotations

from pathlib import Path
import csv
import argparse
import sys

PROJECT_ROOT = Path(__file__).resolve().parents[1]
if str(PROJECT_ROOT) not in sys.path:
    sys.path.insert(0, str(PROJECT_ROOT))

from utils.config_loader import load_base_config
from utils.io import ensure_dir, save_results
from utils.plotting import plot_all
from models.full_model.simulator import run_simulation
from optimization.metrics import extract_summary_metrics
from optimization.iterated.uncertainty import (
    load_parameter_priors,
    run_uncertainty_scan,
    save_rows,
)
from models.calibration.wetlab_bridge import (
    load_wetlab_observations,
    residual_table,
    weighted_rmse,
)
from models.calibration.parameter_fit import (
    random_search_fit,
    save_fit_config,
    save_fit_results,
)
from models.editing.offtarget_panel import apply_offtarget_panel_to_config
from models.editing.sequence_to_kinetics import (
    apply_design_to_config,
    load_design_table,
    pareto_front,
)


def write_summary(summary: dict, path: Path) -> None:
    with open(path, "w", encoding="utf-8", newline="") as f:
        writer = csv.writer(f)
        writer.writerow(["metric", "value"])
        for key, value in summary.items():
            writer.writerow([key, value])


def write_residuals(rows: list[dict], path: Path) -> None:
    if not rows:
        return
    with open(path, "w", encoding="utf-8", newline="") as f:
        writer = csv.DictWriter(f, fieldnames=list(rows[0].keys()))
        writer.writeheader()
        writer.writerows(rows)


```





```python
def write_key_value_table(data: dict, path: Path) -> None:
    with open(path, "w", encoding="utf-8", newline="") as f:
        writer = csv.writer(f)
        writer.writerow(["key", "value"])
        for key, value in data.items():
            writer.writerow([key, value])


def save_dict_rows(rows: list[dict], path: Path) -> None:
    if not rows:
        return
    fieldnames = sorted({key for row in rows for key in row.keys()})
    with open(path, "w", encoding="utf-8", newline="") as f:
        writer = csv.DictWriter(f, fieldnames=fieldnames)
        writer.writeheader()
        writer.writerows(rows)


def utility(summary: dict, weights: dict | None = None) -> float:
    weights = weights or {
        "apoe3": 1.0,
        "apoe2_risk": 0.7,
        "off": 1.2,
        "local": 0.6,
        "puf": 1.0,
        "deam": 1.0,
        "burden": 0.15,
        "late_time": 0.001,
        "reference_time_h": 48.0,
    }
    sample_time = float(summary.get("_sample_time_h", weights["reference_time_h"]))
    late_time_penalty = weights["late_time"] * max(
        sample_time - weights["reference_time_h"], 0.0
    )
    return (
        weights["apoe3"] * float(summary.get("apoe3_like_fraction_final", 0.0))
        - weights["apoe2_risk"] * float(summary.get("ldlr_binding_risk_proxy_final", 0.0))
        - weights["off"] * float(summary.get("off_target_burden_final", 0.0))
        - weights["local"] * float(summary.get("local_bystander_burden_final", 0.0))
        - weights["puf"] * float(summary.get("puf_mismatch_burden_final", 0.0))
        - weights["deam"] * float(summary.get("deaminase_background_burden_final", 0.0))
        - weights["burden"] * float(summary.get("P_brain_peak", 0.0))
        - late_time_penalty
    )


def run_design_screen(
    base_config: dict, design_path: Path, dose: float, dt: float
) -> list[dict]:
    dose_levels = {
        "low": 0.5 * dose,
        "medium": dose,
        "high": 2.0 * dose,
    }
    sample_times = [24.0, 48.0, 72.0]
    rows = []
    for design in load_design_table(design_path):
        design_config, modifiers = apply_design_to_config(base_config, design)
        for dose_label, design_dose in dose_levels.items():
            for sample_time in sample_times:
                sim = run_simulation(
```





```
                    config=design_config,
                    dose=design_dose,
                    t_end=sample_time,
                    dt=dt,
                )
                summary = extract_summary_metrics(sim)
                summary["_sample_time_h"] = sample_time
                rows.append(
                    {
                        "design_id": design["design_id"],
                        "editor_type": design["editor_type"],
                        "dose_level": dose_label,
                        "dose": design_dose,
                        "sample_time_h": sample_time,
                        "utility": utility(summary),
                        **modifiers,
                        **summary,
                    }
                )
    rows.sort(key=lambda row: row["utility"], reverse=True)
    return rows


def write_design_recommendation(rows: list[dict], path: Path) -> None:
    front = pareto_front(rows)
    front_ids = {
        (
            row["design_id"],
            row["dose_level"],
            row["sample_time_h"],
        )
        for row in front
    }
    with open(path, "w", encoding="utf-8") as f:
        f.write("# v4 design recommendation\n\n")
        if rows:
            best = rows[0]
            f.write("## Recommended design\n\n")
            f.write(f"- Design: `{best['design_id']}`\n")
            f.write(f"- Editor: `{best['editor_type']}`\n")
            f.write(f"- Plasmid/expression level: `{best['dose_level']}`\n")
            f.write(f"- Sampling time: `{best['sample_time_h']} h`\n")
            f.write(f"- Utility: `{best['utility']:.4g}`\n")
            f.write(
                f"- Expected APOE4 fraction: `{best.get('apoe4_fraction_final', 0.0):.4g}`\n"
            )
            f.write(
                f"- Expected APOE3-like fraction: `{best.get('apoe3_like_fraction_final', 0.0):.4g
}`\n"
            )
            f.write(
                f"- Expected APOE2-like risk proxy: `{best.get('apoe2_like_fraction_final', 0.0):.
4g}`\n"
            )
            f.write(
                f"- Off-target burden: `{best.get('off_target_burden_final', 0.0):.4g}`\n\n"
            )
        f.write("## Top design rows\n\n")
        f.write(
            "| rank | design | editor | level | time_h | utility | APOE3-like | APOE2-risk | off-t
arget | Pareto |\n"
```





```
        )
        f.write("|---:|---|---|---|---:|---:|---:|---:|---:|---:|---|\n")
        for rank, row in enumerate(rows[:12], start=1):
            key = (row["design_id"], row["dose_level"], row["sample_time_h"])
            is_front = "yes" if key in front_ids else "no"
            f.write(
                f"| {rank} | `{row['design_id']}` | `{row['editor_type']}` | `{row['dose_level']}`
 | "
                f"{row['sample_time_h']:.0f} | {row['utility']:.4g} | "
                f"{row.get('apoe3_like_fraction_final', 0.0):.4g} | "
                f"{row.get('apoe2_like_fraction_final', 0.0):.4g} | "
                f"{row.get('off_target_burden_final', 0.0):.4g} | {is_front} |\n"
            )


def _pearson(xs: list[float], ys: list[float]) -> float:
    if len(xs) < 3:
        return 0.0
    x_mean = sum(xs) / len(xs)
    y_mean = sum(ys) / len(ys)
    numerator = sum((x - x_mean) * (y - y_mean) for x, y in zip(xs, ys))
    x_var = sum((x - x_mean) ** 2 for x in xs)
    y_var = sum((y - y_mean) ** 2 for y in ys)
    if x_var <= 0 or y_var <= 0:
        return 0.0
    return numerator / ((x_var * y_var) ** 0.5)


def write_experimental_design_recommendations(rows: list[dict], path: Path) -> None:
    """
    Rank uncertain parameters by influence on efficacy/safety metrics.

    This is deliberately simple and transparent for iGEM documentation. It is
    not a replacement for global sensitivity analysis, but it is enough to make
    the next wet-lab round model-informed.
    """
    metrics = [
        "on_target_editing_rate_final",
        "off_target_burden_final",
        "specificity_index_final",
        "P_brain_peak",
        "AUC_liver",
        "Cmax_blood",
    ]
    parameter_keys = [key for key in rows[0].keys() if "." in key and key not in metrics]

    rankings = []
    for parameter in parameter_keys:
        xs = [
            float(row[parameter]) for row in rows if row.get(parameter) not in ("", None)
        ]
        if len(xs) != len(rows):
            continue
        score = 0.0
        influences = {}
        for metric in metrics:
            ys = [float(row[metric]) for row in rows]
            corr = _pearson(xs, ys)
            influences[metric] = corr
            score += abs(corr)
        rankings.append((score, parameter, influences))
```





```
    rankings.sort(reverse=True)

    assay_map = {
        "distribution.k_blood_to_liver": "liver vector-genome qPCR/ddPCR time course",
        "distribution.k_blood_to_brain": "brain vector-genome qPCR/ddPCR and tissue fractionation"
,
        "bbb.k_brainblood_to_EC": "brain endothelial versus parenchymal vector localization",
        "bbb.k_endo_to_ISF": "brain parenchymal delivery assay after perfusion",
        "intracellular.k_tx": "PUF-APOBEC mRNA qPCR time course",
        "intracellular.k_deg_m": "actinomycin-D or washout mRNA stability assay",
        "intracellular.k_tl": "tagged editor fluorescence or Western blot time course",
        "intracellular.k_deg_p": "cycloheximide chase or pulse-chase editor stability assay",
        "editing.k_on_on": "EMSA/SPR or early-time on-target editing kinetics",
        "editing.k_off_on": "EMSA/SPR dissociation or editing washout experiment",
        "editing.k_cat_on": "amplicon sequencing editing time course at fixed editor level",
        "editing.k_on_off": "RNA-seq/amplicon off-target panel binding/editing screen",
        "editing.k_cat_off": "off-target amplicon time course for top predicted sites",
    }

    with open(path, "w", encoding="utf-8") as f:
        f.write("# Model-informed next wet-lab experiments\n\n")
        f.write(
            "Parameters are ranked by summed absolute Pearson correlation across key decision metr
ics.\n\n"
        )
        f.write("| rank | parameter | score | recommended experiment |\n")
        f.write("|---:|---|---:|---|\n")
        for rank, (score, parameter, _influences) in enumerate(rankings[:10], start=1):
            experiment = assay_map.get(parameter, "targeted calibration experiment")
            f.write(f"| {rank} | `{parameter}` | {score:.3f} | {experiment} |\n")


def main() -> None:
    parser = argparse.ArgumentParser(
        description="Run the iterated wetlab-integrated model analysis."
    )
    parser.add_argument("--dose", type=float, default=1.0)
    parser.add_argument("--t_end", type=float, default=72.0)
    parser.add_argument("--dt", type=float, default=0.2)
    parser.add_argument("--uncertainty_samples", type=int, default=24)
    parser.add_argument("--fit_samples", type=int, default=24)
    parser.add_argument("--seed", type=int, default=7)
    parser.add_argument(
        "--observations",
        type=str,
        default="wetlab/templates/assay_observations_template.csv",
        help="CSV with wet-lab observations or placeholder template values.",
    )
    parser.add_argument(
        "--offtarget_panel",
        type=str,
        default="wetlab/templates/offtarget_candidate_panel_template.csv",
        help="Candidate off-target panel to aggregate into S_off/k_on_off/k_cat_off.",
    )
    parser.add_argument(
        "--design_table",
        type=str,
        default="wetlab/templates/puf_design_candidates_template.csv",
        help="PUF-deaminase design candidate table for v4 Pareto/utility screening.",
    )
    parser.add_argument("--output_dir", type=str, default="reports/iterated_analysis")
```





```
    args = parser.parse_args()

    project_root = PROJECT_ROOT
    config = load_base_config(project_root)
    out = ensure_dir(project_root / args.output_dir)

    panel_path = project_root / args.offtarget_panel
    if panel_path.exists():
        config, panel_summary = apply_offtarget_panel_to_config(config, panel_path)
        write_key_value_table(panel_summary, out / "offtarget_panel_summary.csv")

    design_path = project_root / args.design_table
    if design_path.exists():
        design_rows = run_design_screen(config, design_path, args.dose, args.dt)
        save_dict_rows(design_rows, out / "v4_design_screen.csv")
        write_design_recommendation(design_rows, out / "v4_design_recommendation.md")

    baseline = run_simulation(config=config, dose=args.dose, t_end=args.t_end, dt=args.dt)
    save_results(baseline, out / "baseline")
    plot_all(baseline, out / "baseline")

    summary = extract_summary_metrics(baseline)
    write_summary(summary, out / "baseline_summary.csv")

    obs_path = project_root / args.observations
    observations = []
    if obs_path.exists():
        observations = load_wetlab_observations(obs_path)
        rows = residual_table(baseline, observations)
        write_residuals(rows, out / "wetlab_residuals.csv")
        with open(out / "wetlab_objective.txt", "w", encoding="utf-8") as f:
            f.write(f"weighted_rmse={weighted_rmse(baseline, observations):.6g}\n")

    priors = load_parameter_priors(project_root / "config" / "parameter_provenance.yaml")
    if observations and args.fit_samples > 0:
        fitted_config, fit_rows = random_search_fit(
            base_config=config,
            priors=priors,
            observations=observations,
            n_samples=args.fit_samples,
            dose=args.dose,
            t_end=args.t_end,
            dt=args.dt,
            seed=args.seed + 101,
        )
        save_fit_results(fit_rows, out / "parameter_fit_results.csv")
        save_fit_config(fitted_config, out / "best_fit_config.yaml")

        fitted = run_simulation(
            config=fitted_config, dose=args.dose, t_end=args.t_end, dt=args.dt
        )
        save_results(fitted, out / "fitted")
        plot_all(fitted, out / "fitted")
        fitted_summary = extract_summary_metrics(fitted)
        write_summary(fitted_summary, out / "fitted_summary.csv")

    uncertainty_rows = list(
        run_uncertainty_scan(
            base_config=config,
            priors=priors,
            n_samples=args.uncertainty_samples,
```





```
            dose=args.dose,
            t_end=args.t_end,
            dt=args.dt,
            seed=args.seed,
        )
    )
    save_rows(uncertainty_rows, out / "uncertainty_samples.csv")
    if uncertainty_rows:
        write_experimental_design_recommendations(
            uncertainty_rows,
            out / "experimental_design_recommendations.md",
        )

    print("Iterated analysis finished.")
    print(f"Output directory: {out}")
    print(
        f"Baseline on-target editing final: {summary['on_target_editing_rate_final']:.6g}"
    )
    print(f"Baseline off-target burden final: {summary['off_target_burden_final']:.6g}")
    print(f"Baseline specificity index final: {summary['specificity_index_final']:.6g}")


if __name__ == "__main__":
    main()
```





benchmarks/run_benchmarks.py

```
"""
CU-REWIRE benchmark reproduction suite (M5).

Reproduces eight published behaviors of PUF-APOBEC rewiring editors
(Han et al. NAR 2022 "CU-REWIRE"; Han et al. Nat Commun 2025 "ProAPOBEC/CU5.x")
in HEK293T transient-transfection mode, each with an explicit quantitative
target, tolerance and PASS / PARTIAL / FAIL verdict.

Unlike a demonstration-only reproduction, every benchmark reports:
  - the numeric model value,
  - the literature-derived target range,
  - the error / ratio actually achieved,
  - a test-type label separating parameterization-consistency checks
    (the model was tuned to this regime) from independent checks
    (emergent directions or quantitative ratios that were NOT fitted).

Usage:
    python benchmarks/run_benchmarks.py --out_dir <abs path>/new_results/benchmarks
"""

from __future__ import annotations

import argparse
import csv
import math
import sys
from copy import deepcopy
from pathlib import Path

ROOT = Path(__file__).resolve().parents[1]
sys.path.insert(0, str(ROOT))

from utils.config_loader import load_base_config  # noqa: E402
from models.full_model.simulator import run_simulation  # noqa: E402
from models.editing.module5 import compute_editing_metrics  # noqa: E402
from models.editing.sequence_to_kinetics import (  # noqa: E402
    load_design_table,
    apply_design_to_config,
)

T_ASSAY = 48.0  # h, standard HEK293T editing readout (Han 2022)
DT = 0.2
DESIGN_TABLE = ROOT / "wetlab" / "templates" / "puf_design_candidates_template.csv"


def run_final_metrics(base_cfg, row, dose, t_end=T_ASSAY):
    """Apply a design row, run transfection-mode simulation, return final metrics."""
    cfg, _ = apply_design_to_config(base_cfg, row)
    cfg["route"] = "transfection"
    res = run_simulation(cfg, dose=dose, t_end=t_end, dt=DT)
    if not res.get("success", False):
        raise RuntimeError(f"solver failed: {res.get('message')}")
    m = compute_editing_metrics(res["y"], res["idx"], cfg)
    out = {k: float(v[-1]) for k, v in m.items()}
    out.update(sequencing_metrics(res["y"], res["idx"]))
    return out
```





```
57
58
59  def sequencing_metrics(y, idx):
60      """
61      Assay-matched metrics: amplicon NGS (as used by Han et al.) classifies
62      every APOE transcript by its edit state, including transcripts transiently
63      bound in editor complexes. The native pipeline metrics instead count only
64      FREE RNA pools in the numerator, which under-reports editing when editor
65      is in vast excess (HEK293T overexpression sequesters product RNA in
66      complexes). Both views are reported; benchmarks B1-B5/B8 use this
67      sequencing view, B6/B7 use accumulated-burden views.
68      """
69
70      def g(name):
71          return float(y[idx[name], -1])
72
73      s4 = g("S_APOE4") + g("C_APOE112") + g("C_APOE158")  # unedited
74      s3 = g("S_APOE3_like") + g("C_APOE158_after112")  # 112 edited
75      s2 = g("S_APOE2_like")  # 112+158 edited
76      s158 = g("S_APOE158_only") + g("C_APOE112_after158")  # 158 only
77      total = s4 + s3 + s2 + s158 + 1e-12
78      return {
79          "seq_edited_112_fraction": (s3 + s2) / total,
80          "seq_apoe2_like_fraction": s2 / total,
81          "seq_158_only_fraction": s158 / total,
82          "seq_unedited_fraction": s4 / total,
83      }
84
85
86  def vary(row, **overrides):
87      r = dict(row)
88      r.update(overrides)
89      return r
90
91
92  def verdict_pass(value, lo, hi):
93      return lo <= value <= hi
94
95
96  def main() -> None:
97      parser = argparse.ArgumentParser(description="CU-REWIRE benchmark reproduction suite")
98      parser.add_argument("--out_dir", type=str, required=True)
99      args = parser.parse_args()
100     out_dir = Path(args.out_dir)
101     out_dir.mkdir(parents=True, exist_ok=True)
102
103     base_cfg = load_base_config(ROOT)
104     rows = {r["design_id"]: r for r in load_design_table(DESIGN_TABLE)}
105     d4 = rows["10R-design-4"]
106
107     results = []
108
109     def add(bid, name, anchor, quantity, value, target_text, status, test_type, detail):
110         results.append(
111             {
112                 "benchmark_id": bid,
113                 "benchmark": name,
114                 "literature_anchor": anchor,
115                 "model_quantity": quantity,
116                 "model_value": f"{value:.6g}",
117                 "target": target_text,
```





```
                "status": status,
                "test_type": test_type,
                "detail": detail,
            }
        )
        print(f"[{status:7s}] {bid} {name}: value={value:.4g} target={target_text}")

    # ---------------------------------------------------------------------- B1
    # Inactive deaminase (E72A-like) control: no detectable editing.
    m = run_final_metrics(base_cfg, rows["inactive-control"], dose=1.0)
    v = m["seq_edited_112_fraction"]
    status = "PASS" if v < 1e-3 else ("PARTIAL" if v < 1e-2 else "FAIL")
    add(
        "B1",
        "Inactive-deaminase control yields no editing",
        "Han 2022: E72A inactive A3A -> no detectable editing",
        "edited-112 fraction (NGS view) @48h",
        v,
        "< 0.1%",
        status,
        "independent negative control",
        f"off-target burden={m['off_target_burden']:.3g}; bg={m['deaminase_background_burden']:.3g
}",
    )

    # ---------------------------------------------------------------------- B2
    # PUF-only control (no active deaminase): no editing.
    m = run_final_metrics(base_cfg, rows["puf-only-control"], dose=1.0)
    v = m["seq_edited_112_fraction"]
    status = "PASS" if v < 1e-3 else ("PARTIAL" if v < 1e-2 else "FAIL")
    add(
        "B2",
        "PUF-only control yields no editing",
        "Han 2022: PUF domain alone -> no editing",
        "edited-112 fraction (NGS view) @48h",
        v,
        "< 0.1%",
        status,
        "independent negative control",
        f"off-target burden={m['off_target_burden']:.3g}",
    )

    # ---------------------------------------------------------------------- B3
    # Preferred design (+2 window, UC motif, 10R) reaches tens of percent.
    m_d4 = run_final_metrics(base_cfg, d4, dose=1.0)
    v = m_d4["seq_edited_112_fraction"]
    if verdict_pass(v, 0.20, 1.00):
        status = "PASS"
    elif verdict_pass(v, 0.05, 1.00):
        status = "PARTIAL"
    else:
        status = "FAIL"
    add(
        "B3",
        "Preferred design reaches tens-of-percent editing",
        "Han 2022: EGFP C504 up to ~60%, CTNNB1 20-45%; Han 2025: CU5.1 APOE c.526 ~90%",
        "edited-112 fraction (NGS view) @48h",
        v,
        "20-100%",
        status,
        "parameterization-consistency",
```





```
        f"APOE2-like={m_d4['seq_apoe2_like_fraction']:.4f}; free-RNA on-target={m_d4['on_target_ed
iting_rate']:.4f}",
    )

    # ------------------------------------------------------------------ B4
    # Editing window: moving the target C from +2 to +7 collapses editing.
    m_w7 = run_final_metrics(
        base_cfg, vary(d4, design_id="d4-win7", distance_112=7.0), dose=1.0
    )
    e2 = max(m_d4["seq_edited_112_fraction"], 1e-12)
    v = m_w7["seq_edited_112_fraction"] / e2
    status = "PASS" if v < 0.2 else ("PARTIAL" if v < 0.5 else "FAIL")
    add(
        "B4",
        "Activity window collapses outside +2 position",
        "Han 2022: editable window centered on 2nd nt downstream of PUF site",
        "editing(+7) / editing(+2)",
        v,
        "< 0.20",
        status,
        "encoded-rule consistency",
        f"editing(+7)={m_w7['seq_edited_112_fraction']:.4f} vs editing(+2)={e2:.4f}",
    )

    # ------------------------------------------------------------------ B5
    # UC vs CC motif preference. Literature ratio 5-18x; model ratio is
    # EMERGENT (saturation compresses the encoded 25x kcat ratio).
    m_cc = run_final_metrics(
        base_cfg, vary(d4, design_id="d4-cc", uc_context_112=0.0), dose=1.0
    )
    e_cc = max(m_cc["seq_edited_112_fraction"], 1e-12)
    v = e2 / e_cc
    if verdict_pass(v, 5.0, 18.0):
        status = "PASS"
    elif verdict_pass(v, 2.5, 25.0):
        status = "PARTIAL"
    else:
        status = "FAIL"
    add(
        "B5",
        "UC-context preference over CC-context",
        "Han 2022: UCN 50.4-67.9% vs CCN 2.8-11.6% editing (ratio ~5-18x)",
        "editing(UC) / editing(CC)",
        v,
        "5-18x",
        status,
        "independent quantitative check (not fitted)",
        f"UC={e2:.4f}, CC={e_cc:.4f}; encoded kcat ratio=25x, saturation compresses output",
    )

    # ------------------------------------------------------------------ B6
    # 10-repeat PUF is more specific than 8-repeat: PUF-dependent off-target
    # (mismatch) burden ratio. Total burden includes the deaminase background
    # shared by both designs, which dilutes the PUF-specific signal, so the
    # PUF-mismatch branch is the quantity the Han 2022 8R->10R claim addresses.
    m_8r = run_final_metrics(base_cfg, rows["8R-design-2"], dose=1.0)
    b10 = max(m_d4["puf_mismatch_burden"], 1e-12)
    v = m_8r["puf_mismatch_burden"] / b10
    if v >= 3.0:
        status = "PASS"
    elif v >= 1.5:
```





```
        status = "PARTIAL"
    else:
        status = "FAIL"
    add(
        "B6",
        "10R PUF more specific than 8R PUF",
        "Han 2022: extending PUF recognition 8R->10R strongly reduces off-target editing",
        "PUF-mismatch burden(8R) / (10R)",
        v,
        ">= 3x",
        status,
        "direction test (magnitude partially encoded)",
        f"8R puf-mismatch={m_8r['puf_mismatch_burden']:.5f}, 10R={b10:.5f}; "
        f"total-burden ratio={m_8r['off_target_burden']/max(m_d4['off_target_burden'],1e-12):.2f}x
",
    )

    # ------------------------------------------------------------------ B7
    # Free-deaminase background: >0 for active editor, scales with expression,
    # exactly zero for inactive control.
    m_lo = run_final_metrics(base_cfg, d4, dose=0.5)
    m_hi = run_final_metrics(base_cfg, d4, dose=2.0)
    m_in = run_final_metrics(base_cfg, rows["inactive-control"], dose=2.0)
    bg_lo = m_lo["deaminase_background_burden"]
    bg_hi = m_hi["deaminase_background_burden"]
    bg_in = m_in["deaminase_background_burden"]
    v = bg_hi / max(bg_lo, 1e-12)
    ok = (bg_hi > bg_lo > 0.0) and (bg_in == 0.0)
    status = "PASS" if ok else ("PARTIAL" if bg_hi > 0 and bg_in == 0.0 else "FAIL")
    add(
        "B7",
        "Deaminase background scales with expression, zero when inactive",
        "Han 2022: free A3A gives dose-dependent background editing (~20% unfused)",
        "bg(dose 2.0) / bg(dose 0.5)",
        v,
        "> 1, bg(inactive)=0",
        status,
        "direction test",
        f"bg(0.5)={bg_lo:.4f}, bg(2.0)={bg_hi:.4f}, bg(inactive)={bg_in:.3g}",
    )

    # ------------------------------------------------------------------ B8
    # Time course: editing accumulates monotonically across 24/48/72 h.
    ts = [24.0, 48.0, 72.0]
    e_ts = [
        run_final_metrics(base_cfg, d4, dose=1.0, t_end=t)["seq_edited_112_fraction"]
        for t in ts
    ]
    mono = (
        all(e_ts[i + 1] >= e_ts[i] - 1e-9 for i in range(len(ts) - 1))
        and e_ts[-1] > e_ts[0] + 1e-3
    )
    v = e_ts[-1]
    status = "PASS" if mono else "FAIL"
    add(
        "B8",
        "Editing accumulates monotonically over 24/48/72 h",
        "Han 2022: editing increases with assay duration and dose",
        "edited-112 fraction @72h",
        v,
        "monotone increasing",
```





```
        status,
        "consistency check",
        f"24h={e_ts[0]:.4f}, 48h={e_ts[1]:.4f}, 72h={e_ts[2]:.4f}",
    )

    # ---------------------------------------------------------------------- out
    n_pass = sum(1 for r in results if r["status"] == "PASS")
    n_part = sum(1 for r in results if r["status"] == "PARTIAL")
    n_fail = sum(1 for r in results if r["status"] == "FAIL")
    summary = {
        "n_benchmarks": len(results),
        "n_pass": n_pass,
        "n_partial": n_part,
        "n_fail": n_fail,
        "score": f"{n_pass}/{len(results)} PASS, {n_part} PARTIAL, {n_fail} FAIL",
        "time_course_24_48_72": e_ts,
    }

    csv_path = out_dir / "benchmark_scorecard.csv"
    with open(csv_path, "w", newline="", encoding="utf-8") as f:
        writer = csv.DictWriter(f, fieldnames=list(results[0].keys()))
        writer.writeheader()
        writer.writerows(results)

    md_path = out_dir / "benchmark_scorecard.md"
    with open(md_path, "w", encoding="utf-8") as f:
        f.write("# CU-REWIRE benchmark scorecard (model v4, transfection mode)\n\n")
        f.write(f"**Score: {summary['score']}**\n\n")
        f.write(
            "| ID | Benchmark | Literature anchor | Model value | Target | Status | Test type |\n"
        )
        f.write(
            "|----|-----------|-------------------|-------------|--------|--------|-----------|\n"
        )
        for r in results:
            f.write(
                f"| {r['benchmark_id']} | {r['benchmark']} | {r['literature_anchor']} "
                f"| {r['model_value']} | {r['target']} | {r['status']} | {r['test_type']} |\n"
            )
        f.write("\n")
        for r in results:
            f.write(f"- **{r['benchmark_id']}** {r['detail']}\n")

    import json

    with open(out_dir / "benchmark_summary.json", "w", encoding="utf-8") as f:
        json.dump(summary, f, indent=2)

    print(f"\nScore: {summary['score']}")
    print(f"CSV: {csv_path}")
    print(f"MD:  {md_path}")


if __name__ == "__main__":
    main()
```





run_module6_scan.py

```python
from pathlib import Path

from utils.config_loader import load_base_config
from optimization.min_effective_dose import run_dose_scan, find_min_effective_dose
from optimization.report_module6 import save_dose_scan_table, plot_dose_response


def main():
    project_root = Path(__file__).resolve().parent
    config = load_base_config(project_root)

    scan_results = run_dose_scan(base_config=config)
    best = find_min_effective_dose(scan_results)

    output_dir = project_root / "outputs" / "run_module6"
    csv_path = save_dose_scan_table(scan_results, output_dir)
    fig_paths = plot_dose_response(scan_results, output_dir)

    print(f"Dose scan saved to: {csv_path}")
    if best is None:
        print("No feasible dose found under current thresholds.")
    else:
        print("Minimum feasible dose found:")
        for k, v in best.items():
            print(f"  {k}: {v}")

    for name, path in fig_paths.items():
        print(f"{name}: {path}")


if __name__ == "__main__":
    main()
```





run_sensitivity_analysis.py

```python
from __future__ import annotations

from pathlib import Path
import copy
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from utils.config_loader import load_base_config
from models.full_model.simulator import run_simulation
from optimization.metrics import extract_summary_metrics


PARAMETER_MAP = {
    "k_lymph": ("absorption", "k_lymph"),
    "k_blood": ("absorption", "k_blood"),
    "k_deg_loc": ("absorption", "k_deg_loc"),
    "k_blood_to_liver": ("distribution", "k_blood_to_liver"),
    "k_blood_to_brain": ("distribution", "k_blood_to_brain"),
    "k_clear_blood": ("distribution", "k_clear_blood"),
    "k_endo_to_ISF": ("bbb", "k_endo_to_ISF"),
    "k_endo_loss": ("bbb", "k_endo_loss"),
    "k_ISF_to_cell": ("intracellular", "k_ISF_to_cell"),
    "k_cell_to_nuc": ("intracellular", "k_cell_to_nuc"),
    "k_tx": ("intracellular", "k_tx"),
    "k_tl": ("intracellular", "k_tl"),
    "k_cat_on": ("editing", "k_cat_on"),
    "k_cat_off": ("editing", "k_cat_off"),
}


OUTPUT_METRICS = [
    "P_brain_peak",
    "on_target_editing_rate_final",
    "off_target_burden_final",
    "specificity_index_final",
    "AUC_liver",
    "Cmax_blood",
]


def get_param_block(config: dict, block_name: str) -> dict:
    if block_name == "absorption":
        route = config["route"]
        return config["absorption"][route]
    return config[block_name]


def set_parameter(config: dict, param_name: str, scale: float) -> tuple[float, float]:
    block_name, key = PARAMETER_MAP[param_name]
    block = get_param_block(config, block_name)
    old_value = block[key]
    block[key] = old_value * scale
    return old_value, block[key]


```





```python
def run_model_and_extract(config: dict, dose: float, t_end: float, dt: float) -> dict:
    results = run_simulation(config=config, dose=dose, t_end=t_end, dt=dt)
    return extract_summary_metrics(results)


def compute_normalized_sensitivity(
    y_base: float, y_pert: float, p_base: float, p_pert: float
) -> float:
    if y_base == 0 or p_base == 0:
        return np.nan
    return ((y_pert - y_base) / y_base) / ((p_pert - p_base) / p_base)


def main():
    # 因为脚本在项目根目录，所以用 .parent
    project_root = Path(__file__).resolve().parent
    config = load_base_config(project_root)

    dose = 1.0
    t_end = 168.0
    dt = 0.2

    output_dir = project_root / "outputs" / "sensitivity_analysis"
    output_dir.mkdir(parents=True, exist_ok=True)

    base_metrics = run_model_and_extract(
        copy.deepcopy(config), dose=dose, t_end=t_end, dt=dt
    )

    rows = []

    for param_name in PARAMETER_MAP:
        for scale in [0.8, 1.2]:
            cfg = copy.deepcopy(config)
            p_base, p_pert = set_parameter(cfg, param_name, scale)
            pert_metrics = run_model_and_extract(cfg, dose=dose, t_end=t_end, dt=dt)

            row = {
                "parameter": param_name,
                "scale": scale,
                "p_base": p_base,
                "p_pert": p_pert,
            }

            for metric in OUTPUT_METRICS:
                row[f"{metric}_base"] = base_metrics[metric]
                row[f"{metric}_pert"] = pert_metrics[metric]
                row[f"{metric}_sens"] = compute_normalized_sensitivity(
                    base_metrics[metric],
                    pert_metrics[metric],
                    p_base,
                    p_pert,
                )

            rows.append(row)

    df = pd.DataFrame(rows)
    csv_path = output_dir / "local_sensitivity_results.csv"
    df.to_csv(csv_path, index=False)

    # 画图：用 +20% 扰动的归一化敏感度
```





```
    df_plot = df[df["scale"] == 1.2].copy()

    for metric in OUTPUT_METRICS:
        plot_df = df_plot[["parameter", f"{metric}_sens"]].copy()
        plot_df["abs_sens"] = plot_df[f"{metric}_sens"].abs()
        plot_df = plot_df.sort_values("abs_sens", ascending=False)

        plt.figure(figsize=(9, 5))
        plt.bar(plot_df["parameter"], plot_df[f"{metric}_sens"])
        plt.xticks(rotation=45, ha="right")
        plt.ylabel("Normalized sensitivity")
        plt.title(f"Sensitivity of {metric}")
        plt.tight_layout()
        plt.savefig(output_dir / f"sensitivity_{metric}.png", dpi=160)
        plt.close()

    print(f"Sensitivity analysis finished. Results saved to: {output_dir}")
    print(f"CSV: {csv_path}")


if __name__ == "__main__":
    main()
```





smoke_check.py

```python
"""Short software check only; no dose scan, fitting, or uncertainty analysis."""

from pathlib import Path
import os
import runpy
import numpy as np
from models.full_model.simulator import run_simulation
from utils.config_loader import load_base_config


def main():
    root = Path(__file__).resolve().parent
    os.chdir(root)
    test_files = (
        "test_administration_routes.py",
        "test_sequence_to_kinetics.py",
        "test_offtarget_panel.py",
        "test_wetlab_bridge.py",
    )
    passed = []
    for filename in test_files:
        namespace = runpy.run_path(str(root / "tests" / filename))
        for name, function in sorted(namespace.items()):
            if name.startswith("test_") and callable(function):
                function()
                passed.append(name)
    result = run_simulation(load_base_config(root), dose=1.0, t_end=1.0, dt=0.5)
    assert result["success"], result["message"]
    assert result["t"].shape == (3,)
    assert result["y"].shape == (len(result["state_order"]), 3)
    assert np.isfinite(result["y"]).all()
    assert result["state_order"] == list(dict.fromkeys(result["state_order"]))
    print(f"PASS: {len(passed)} existing contract tests")
    print(
        f"PASS: 1-hour smoke trajectory, shape={result['y'].shape}, finite values, solver success"
    )
    print("No scientific validation or full scan was performed.")


if __name__ == "__main__":
    main()
```





tests/test_administration_routes.py

```python
from __future__ import annotations

from models.full_model.simulator import build_initial_state
from models.full_model.state_vector import STATE_ORDER, build_index_map
from utils.config_loader import load_base_config


def test_config_supports_all_administration_routes() -> None:
    config = load_base_config(".")

    assert {"footpad", "im", "iv"} <= set(config["absorption"])


def test_iv_dose_starts_in_blood_instead_of_local_depot() -> None:
    config = load_base_config(".")
    config["route"] = "iv"
    idx = build_index_map(STATE_ORDER)

    initial = build_initial_state(2.5, config)

    assert initial[idx["A_dep"]] == 0.0
    assert initial[idx["A_blood"]] == 2.5
```





tests/test_apoe_multisite_model.py

```python
from __future__ import annotations

import copy

from utils.config_loader import load_base_config
from models.full_model.simulator import run_simulation
from models.editing.module5 import compute_editing_metrics


def test_apoe_multisite_model_produces_site_specific_fractions():
    config = load_base_config(".")
    config = copy.deepcopy(config)
    config["editing"]["model"] = "apoe_multisite"

    results = run_simulation(config=config, dose=1.0, t_end=24.0, dt=1.0)
    metrics = compute_editing_metrics(results["y"], results["idx"], config)

    assert results["success"]
    assert "apoe4_fraction" in metrics
    assert "apoe3_like_fraction" in metrics
    assert "apoe2_like_fraction" in metrics
    assert "local_bystander_burden" in metrics
    assert metrics["apoe3_like_fraction"][-1] >= 0.0
    assert metrics["apoe2_like_fraction"][-1] >= 0.0
```





tests/test_brain_app.py

```python
from __future__ import annotations

import io

from fastapi.testclient import TestClient

from brain_app.app import create_app
from brain_app.service import MAX_PANEL_BYTES, parse_candidate_panel, run_model


def test_home_exposes_model_inputs_and_disclaimer() -> None:
    client = TestClient(create_app())

    response = client.get("/")

    assert response.status_code == 200
    assert "Brain Delivery Model" in response.text
    assert "Single simulation" in response.text
    assert "Dose optimization" in response.text
    assert "not clinically calibrated" in response.text
    assert client.get("/assets/plotly.js").status_code == 200


def test_candidate_panel_parser_enforces_contract() -> None:
    valid = (
        "site_id,gene,initial_pool,binding_score,accessibility,context_score,"
        "validation_priority,notes\n"
        "puf-1,APOE,0.8,0.9,0.7,0.6,High priority,synthetic\n"
    ).encode()

    rows = parse_candidate_panel(valid)

    assert rows[0]["site_id"] == "puf-1"
    assert rows[0]["binding_score"] == 0.9


def test_candidate_panel_parser_rejects_invalid_scores_and_large_uploads() -> None:
    invalid = (
        "site_id,gene,initial_pool,binding_score,accessibility,context_score,"
        "validation_priority\n"
        "bad,APOE,0.8,1.2,0.7,0.6,High\n"
    ).encode()

    try:
        parse_candidate_panel(invalid)
    except ValueError as error:
        assert "binding_score" in str(error)
    else:
        raise AssertionError("invalid score was accepted")

    try:
        parse_candidate_panel(b"x" * (MAX_PANEL_BYTES + 1))
    except ValueError as error:
        assert "5 MB" in str(error)
    else:
        raise AssertionError("oversized panel was accepted")
```





```
57
58
59  def test_single_simulation_api_applies_design_and_candidate_panel() -> None:
60      client = TestClient(create_app())
61      panel = (
62          "site_id,gene,initial_pool,binding_score,accessibility,context_score,"
63          "validation_priority,notes\n"
64          "puf-1,GENE1,0.8,0.9,0.7,0.6,High priority,synthetic\n"
65      ).encode()
66
67      response = client.post(
68          "/api/simulate",
69          data={
70              "mode": "single",
71              "design_id": "10R-design-4",
72              "route": "footpad",
73              "dose": "1.0",
74              "duration": "24",
75          },
76          files={
77              "candidate_panel": (
78                  "brain_candidate_panel.csv",
79                  io.BytesIO(panel),
80                  "text/csv",
81              )
82          },
83      )
84
85      assert response.status_code == 200
86      payload = response.json()
87      assert payload["mode"] == "single"
88      assert payload["inputs"]["design_id"] == "10R-design-4"
89      assert payload["panel_summary"]["n_sites"] == 1.0
90      assert payload["panel_summary"]["applied"] is True
91      assert 0.0 <= payload["metrics"]["apoe3_like_fraction_final"] <= 1.0
92      assert 0.0 <= payload["metrics"]["apoe2_like_fraction_final"] <= 1.0
93      assert len(payload["series"]["time"]) == len(payload["series"]["pbrain"])
94
95
96  def test_dose_optimization_api_returns_twelve_doses() -> None:
97      client = TestClient(create_app())
98
99      response = client.post(
100         "/api/simulate",
101         data={
102             "mode": "optimize",
103             "design_id": "10R-design-4",
104             "route": "im",
105             "dose": "1.0",
106             "duration": "24",
107         },
108     )
109
110     assert response.status_code == 200
111     payload = response.json()
112     assert payload["mode"] == "optimize"
113     assert len(payload["dose_scan"]) == 12
114     assert payload["dose_scan"][0]["dose"] == 0.1
115
116
117 def test_invalid_request_returns_explainable_error() -> None:
```





```
    client = TestClient(create_app())

    response = client.post(
        "/api/simulate",
        data={
            "mode": "single",
            "design_id": "not-a-design",
            "route": "iv",
            "dose": "99",
            "duration": "24",
        },
    )

    assert response.status_code == 422
    assert "dose" in response.json()["detail"].lower()


def test_empty_uploaded_panel_retains_conservative_base_prior() -> None:
    inputs = {
        "mode": "single",
        "design_id": "10R-design-4",
        "route": "footpad",
        "dose": 1.0,
        "duration": 24.0,
    }

    baseline = run_model(**inputs, panel_rows=None)
    empty_panel = run_model(**inputs, panel_rows=[])

    assert empty_panel["metrics"] == baseline["metrics"]
    assert empty_panel["panel_summary"]["provided"] is True
    assert empty_panel["panel_summary"]["applied"] is False
    assert "retained" in empty_panel["panel_summary"]["warning"].lower()


def test_web_metrics_match_direct_service_call() -> None:
    client = TestClient(create_app())
    form = {
        "mode": "single",
        "design_id": "10R-design-4",
        "route": "iv",
        "dose": "0.8",
        "duration": "24",
    }

    response = client.post("/api/simulate", data=form)
    direct = run_model(
        mode="single",
        design_id="10R-design-4",
        route="iv",
        dose=0.8,
        duration=24.0,
    )

    assert response.status_code == 200
    assert response.json()["metrics"] == direct["metrics"]


def test_web_rejects_oversized_panel_before_parsing() -> None:
    client = TestClient(create_app())

```





```
    response = client.post(
        "/api/simulate",
        data={
            "mode": "single",
            "design_id": "10R-design-4",
            "route": "footpad",
            "dose": "1.0",
            "duration": "24",
        },
        files={
            "candidate_panel": ("too-large.csv", b"x" * (MAX_PANEL_BYTES + 1), "text/csv")
        },
    )

    assert response.status_code == 422
    assert "5 MB" in response.json()["detail"]
```





tests/test_offtarget_panel.py

```
from __future__ import annotations

from models.editing.offtarget_panel import summarize_offtarget_panel


def test_offtarget_panel_summary_uses_accessibility_weighting():
    rows = [
        {
            "site_id": "a",
            "gene": "A",
            "initial_pool": 1.0,
            "binding_score": 0.5,
            "accessibility": 1.0,
            "context_score": 0.5,
        },
        {
            "site_id": "b",
            "gene": "B",
            "initial_pool": 1.0,
            "binding_score": 1.0,
            "accessibility": 0.0,
            "context_score": 1.0,
        },
    ]

    summary = summarize_offtarget_panel(rows)

    assert summary["effective_pool"] == 1.0
    assert summary["relative_binding"] == 0.5
    assert summary["relative_catalysis"] == 0.25
```





tests/test_sequence_to_kinetics.py

```python
from __future__ import annotations

from models.editing.sequence_to_kinetics import design_to_kinetic_modifiers


def test_sequence_features_increase_intended_kinetics():
    weak = {
        "design_id": "weak",
        "editor_type": "A3A",
        "puf_repeats": 8,
        "mismatch_count": 2,
        "puf_score_112": 0.2,
        "puf_score_158": 0.1,
        "accessibility_112": 0.2,
        "accessibility_158": 0.2,
        "distance_112": 8,
        "distance_158": 8,
        "uc_context_112": 0.0,
        "uc_context_158": 0.0,
        "local_bystander_risk": 0.1,
        "deaminase_background_prior": 1.0,
    }
    strong = {
        **weak,
        "design_id": "strong",
        "puf_repeats": 10,
        "mismatch_count": 0,
        "puf_score_112": 0.9,
        "accessibility_112": 0.8,
        "distance_112": 2,
        "uc_context_112": 1.0,
    }

    weak_mod = design_to_kinetic_modifiers(weak)
    strong_mod = design_to_kinetic_modifiers(strong)

    assert strong_mod["kon112_scale"] > weak_mod["kon112_scale"]
    assert strong_mod["kcat112_scale"] > weak_mod["kcat112_scale"]
    assert strong_mod["puf_offtarget_scale"] < weak_mod["puf_offtarget_scale"]
```





tests/test_wetlab_bridge.py

```python
from __future__ import annotations

import numpy as np

from models.calibration.wetlab_bridge import simulated_assay_value


def _fake_results():
    state_order = [
        "A_blood",
        "A_liver",
        "A_brain_nuc",
        "mRNA_brain",
        "P_brain",
        "S_on",
        "E_on",
        "S_off",
        "E_off",
    ]
    idx = {name: i for i, name in enumerate(state_order)}
    y = np.array(
        [
            [1.0, 0.5],
            [0.0, 2.0],
            [0.0, 0.1],
            [0.0, 0.2],
            [0.0, 0.3],
            [1.0, 0.8],
            [0.0, 0.2],
            [5.0, 4.9],
            [0.0, 0.1],
        ]
    )
    return {"t": np.array([0.0, 10.0]), "y": y, "idx": idx}


def test_amplicon_percent_observables_are_state_derived():
    results = _fake_results()
    assert (
        abs(simulated_assay_value(results, "on_editing_amplicon_pct", 10.0) - 20.0) < 1e-9
    )
    assert 1.9 < simulated_assay_value(results, "off_editing_amplicon_pct", 10.0) < 2.1


def test_delivery_and_expression_observables_are_available():
    results = _fake_results()
    assert simulated_assay_value(results, "vector_qpcr_blood", 10.0) == 0.5
    assert simulated_assay_value(results, "puf_mrna_qpcr", 10.0) == 0.2
    assert simulated_assay_value(results, "editor_western", 10.0) == 0.3
```





requirements-core-tested.txt

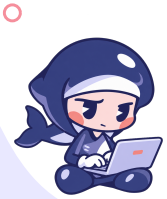
```
# Core versions observed in the smoke-tested Python 3.12.14 runtime.
numpy==2.3.5
scipy==1.18.1
matplotlib==3.10.9
PyYAML==6.0.3
```





requirements.txt

```
numpy
scipy
matplotlib
pyyaml
fastapi
jinja2
plotly
python-multipart
uvicorn
```





wetlab/templates/assay_observations_template.csv

```csv
assay,time_h,value,sd,weight,notes
vector_qpcr_blood,1,0.2,0.05,1,replace with measured blood vector genome proxy
vector_qpcr_liver,24,0.3,0.08,1,replace with liver qPCR/ddPCR proxy
vector_qpcr_brain,72,0.001,0.0003,2,brain delivery is a key decision metric
puf_mrna_qpcr,72,0.001,0.0002,1,PUF-APOBEC transcript expression
editor_western,72,0.003,0.0008,1,protein-level expression proxy
on_editing_amplicon_pct,72,1.3,0.3,3,on-target amplicon sequencing percent
apoe3_like_amplicon_pct,72,1.2,0.3,3,112-only E4-to-E3-like editing percent
apoe2_like_amplicon_pct,72,0.05,0.03,2,double-edit APOE2-like risk proxy
off_editing_amplicon_pct,72,0.03,0.02,3,off-target amplicon sequencing percent
local_bystander_amplicon_pct,72,0.05,0.03,2,nearby C edits around intended APOE target
puf_mismatch_offtarget_pct,72,0.02,0.02,2,PUF-mediated transcriptome off-target panel
deaminase_background_pct,72,0.02,0.02,2,deaminase-only background editing
viability_pct,72,95,5,1,cell viability or burden readout
```





wetlab/templates/offtarget_candidate_panel_template.csv

```
site_id,gene,initial_pool,binding_score,accessibility,context_score,validation_priority,notes
APOE4_target_like_site_001,APOE,1.0,0.20,0.70,0.80,high,placeholder off-target-like site for pipel
ine testing
off_site_002,GENE_A,0.8,0.12,0.50,0.60,medium,replace with REWIRE transcriptome candidate
off_site_003,GENE_B,1.2,0.06,0.30,0.50,low,replace with REWIRE transcriptome candidate
off_site_004,GENE_C,0.5,0.18,0.80,0.70,high,replace with REWIRE transcriptome candidate
```





wetlab/templates/puf_design_candidates_template.csv

```
design_id,editor_type,puf_repeats,target_site,puf_score_112,puf_score_158,mismatch_count,accessibility_112,accessibility_158,distance_112,distance_158,uc_context_112,uc_context_158,local_bystander_risk,deaminase_background_prior,editor_activity_scale,notes
10R-design-4,A3A,10,APOE112,0.95,0.25,0,0.75,0.45,2,7,1.0,0.4,0.04,0.8,1.0,preferred E4-to-E3-like candidate
10R-dual-site,A3A,10,APOE112_APOE158,0.85,0.65,1,0.70,0.55,2,2,1.0,0.8,0.10,0.9,1.0,dual-edit candidate with higher APOE2-like risk
8R-design-2,A3A,8,APOE112,0.70,0.20,1,0.65,0.40,3,8,0.8,0.3,0.08,1.2,1.0,lower-specificity legacy style design
10R-proapobec,ProAPOBEC,10,APOE112,0.90,0.30,0,0.75,0.45,2,6,1.0,0.4,0.03,0.4,1.0,engineered deaminase candidate
inactive-control,A3A,10,control,0.90,0.30,0,0.75,0.45,2,6,0.0,0.0,0.00,0.0,0.0,inactive deaminase control
puf-only-control,A3A,10,control,0.90,0.30,0,0.75,0.45,2,6,0.0,0.0,0.00,0.0,0.0,PUF binding control without active deaminase
```



